B.Tech. BCSE497J - Project-I

PRAESENS: A MULTI-LANE PHYSICAL CHALLENGE–RESPONSE FRAMEWORK FOR REAL-TIME DETECTION OF DEEPFAKE AND VIDEO-INJECTION ATTACKS IN REMOTE FACE VERIFICATION

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering with Specialization in Bioinformatics

by

23BCB0030	**SUHAANI KEDAR**
23BCB0074	**UPASANA BHAUMIK**

Under the Supervision of

DR. BERLIN M A

Associate Professor Senior Grade 2

School of Computer Science and Engineering (SCOPE)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

We hereby declare that the project entitled **“PRAESENS: A Multi-Lane Physical Challenge–Response Framework for Real-Time Detection of Deepfake and Video-Injection Attacks in Remote Face Verification”** submitted by us, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering with Specialization in Bioinformatics* to VIT is a record of bonafide work carried out by us under the supervision of **Dr. Berlin M A**.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore
Date :

Suhaani Kedar (23BCB0030) **Upasana Bhaumik (23BCB0074)**

Signature of the Candidates

CERTIFICATE

This is to certify that the project entitled **"PRAESENS: A Multi-Lane Physical Challenge–Response Framework for Real-Time Detection of Deepfake and Video-Injection Attacks in Remote Face Verification"** submitted by **Suhaani Kedar (23BCB0030)** and **Upasana Bhaumik (23BCB0074)**, School of Computer Science and Engineering, VIT, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering with Specialization in Bioinformatics*, is a record of bonafide work carried out by them under my supervision during Fall Semester 2026-2027, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore
Date :

Signature of the Guide
Dr. Berlin M A

Examiner(s)

Head of the Department
B.Tech. Computer Science and Engineering
with Specialization in Bioinformatics

ACKNOWLEDGEMENTS

We are deeply grateful to the management of Vellore Institute of Technology (VIT) for providing us with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in our academic journey. The support and infrastructure provided by VIT have enabled us to explore and develop our ideas to their fullest potential.

Our sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering (SCOPE), for his unwavering support and encouragement. His leadership and vision have greatly inspired us to strive for excellence. The Dean's dedication to academic excellence and innovation has been a constant source of motivation for us. We appreciate his efforts in creating an environment that nurtures creativity and critical thinking.

We express our profound appreciation to the Head of the Department, B.Tech. Computer Science and Engineering with Specialization in Bioinformatics, for the insightful guidance and continuous support extended to us. The department's commitment to fostering a collaborative and supportive atmosphere has greatly enhanced our learning experience, and its constructive feedback and encouragement have been invaluable in overcoming challenges and achieving our project goals.

We are immensely thankful to our project guide, **Dr. Berlin M A**, Associate Professor Senior Grade 2, SCOPE, for her dedicated mentorship and invaluable feedback. Her patience, knowledge, and encouragement have been pivotal in the successful completion of this project. Her willingness to share her expertise and to push us to measure honestly rather than claim optimistically has been instrumental in refining our ideas and methodologies. Her support has not only contributed to the success of this project but has also enriched our overall academic experience.

Thank you all for your contributions and support.

Suhaani Kedar **Upasana Bhaumik**

TABLE OF CONTENTS

Sl. No	Contents	Page No.
	Abstract	**xii**
1.	**INTRODUCTION**	**1**
	1.1 Background	1
	1.2 Motivations	1
	1.3 Scope of the Project	2
2.	**PROJECT DESCRIPTION AND GOALS**	**4**
	2.1 Literature Review	4
	2.1.1 Face manipulation and the deepfake threat	4
	2.1.2 Passive presentation-attack detection	4
	2.1.3 Physiological-signal methods	5
	2.1.4 Active challenge–response liveness	5
	2.1.5 Sequence design, detection and fusion foundations	5
	2.2 Research Gap	6
	2.3 Objectives	6
	2.4 Problem Statement	7
	2.5 Project Plan	7
3.	**TECHNICAL SPECIFICATION**	**9**
	3.1 Requirements	9
	3.1.1 Functional	9
	3.1.2 Non-Functional	9
	3.2 Feasibility Study	10
	3.2.1 Technical Feasibility	10
	3.2.2 Economic Feasibility	10
	3.2.3 Social Feasibility	10
	3.3 System Specification	11
	3.3.1 Hardware Specification	11
	3.3.2 Software Specification	11
4.	**DESIGN APPROACH AND DETAILS**	**12**
	4.1 System Architecture	12
	4.2 Design	14
	4.2.1 Data Flow Diagram	14

4.2.2 Use Case Diagram 16
4.2.3 Class Diagram 17
4.2.4 Sequence Diagram 19
4.2.5 Verdict Decision Logic 21
5. METHODOLOGY AND TESTING 22
5.1 Module Description 22
5.1.1 Challenge Synthesiser (challenge.py) 22
5.1.2 Emitter (emit.py, spatial.py) 23
5.1.3 Optical Lane (optical.py, capture.py) 24
5.1.4 Spatially-Differential Challenge (spatial.py) 25
5.1.5 Typing Lane (typing.py) 25
5.1.6 Acoustic Lane (acoustic.py) 26
5.1.7 Fusion and Adjudication (fusion.py) 27
5.1.8 Attack Tooling (demo/attack.py, attacks/adaptive_injector.py) 27
5.1.9 Evaluation Pipeline (eval/) 27
5.2 Testing 28
5.2.1 Testing Strategy 28
5.2.2 Representative Test Cases 28
5.2.3 Defects Found Through Testing 29
6. PROJECT DEMONSTRATION 31
6.1 Verification Session 31
6.2 Live Dashboard and Panel Mode 32
6.3 What the Demonstration Shows 33
7. RESULT AND DISCUSSION 34
7.1 Evaluation Corpus 34
7.2 Optical Lane: Separation and Error Rates 34
7.3 Timing as Evidence: Lag Analysis 36
7.4 Real Injection Attacks and Fused Verdicts 36
7.5 Defence in Depth: Spatial Challenge and Ablation 38
7.6 Robustness and Operating Conditions 38
7.7 Comparison with Existing Approaches 40
7.8 Cost Analysis 40
7.9 Limitations 41
8. CONCLUSION 42

8.1 Future Work 42

9. REFERENCES 44

APPENDIX A – SAMPLE CODE 47

LIST OF FIGURES

Figure No.	Title	Page No.
2.1	Taxonomy of attacks on remote face verification and the defence family that addresses each	4
2.2	Project plan (Gantt chart), July – October 2026	8
4.1	Layered system architecture of PRAESENS	13
4.2	Four successive frames rendered by `SpatialEmitter.render_frame()`; each border zone switches independently between B+A and B−A	14
4.3	Level 0 (context) data flow diagram	15
4.4	Level 1 data flow diagram	16
4.5	Use case diagram	17
4.6	Class diagram of the core `praesens` package	18
4.7	Sequence diagram of a concurrent three-lane verification session	20
4.8	Binary-gate adjudication flowchart	21
5.1	Order-6 m-sequence, its ideal periodic autocorrelation, and the cross-correlation of the three derived zone codes (diagonal 1.00, off-diagonal ≤ 0.02)	23
5.2	ROI × zone correlation matrices computed by `compute_correlation_matrix()`: a genuine face is diagonal-dominant (S_{sp} = 0.71) while a global-brightness attack is uniform (S_{sp} = −0.02)	25
5.3	Acoustic probe envelope with a 4 kHz carrier inset, and the segmented differential matched-filter statistic for a simulated 330 ms round trip versus an off-carrier noise channel	26
5.4	Distribution of the 148 automated test cases across modules	28
6.1	Verdict banners: ACCEPT (green, all lanes pass), RE-CHALLENGE (amber, a secondary lane failed), REJECT (red, the optical lane failed)	32
6.2	Real recording from session 20260914T115241: the emitted challenge (orange) and the detrended luminance measured on the user's forehead and cheeks (navy)	33
6.3	Correlation versus hypothesised lag for one real trace scored against its own challenge and against a different session's challenge	33
7.1	Optical score distributions: bona fide sessions separate clearly from the injection proxy, the emitter-off control and real virtual-camera injection	35
7.2	ROC curve of the optical lane	35
7.3	Distribution of the lag at the correlation peak for bona fide and cross-session trials	36
7.4	Optical score versus estimated SNR for every real session	37
7.5	Verdicts of the thirteen fully-adjudicated live sessions	37

7.6	Ablation: fraction of ACCEPT verdicts lost when each lane is removed	38
7.7	Mean bona fide optical score by lighting, eyewear and makeup (error bars: one standard deviation)	39
7.8	Measured capture rate versus manual exposure value, and the chip rate selected automatically	39

LIST OF TABLES

Table No.	Title	Page No.
2.1	Summary of representative literature and its limitations with respect to injection attacks	5
2.2	Milestones and deliverables	8
3.1	Functional requirements	9
3.2	Non-functional requirements	9
3.3	Hardware specification (development and evaluation platform)	11
3.4	Software specification	11
5.1	Representative test cases and outcomes	28
5.2	Significant defects discovered and fixed	29
6.1	Panel-mode controls and what each demonstrates	32
7.1	Real session corpus used for evaluation	34
7.2	Optical score distribution per condition	34
7.3	ISO/IEC 30107-3 metrics at the EER threshold	35
7.4	Confusion summary of fully-adjudicated live sessions	37
7.5	Qualitative comparison with representative approaches	40
7.6	Cost analysis	40

LIST OF ABBREVIATIONS

ACER	Average Classification Error Rate
APCER	Attack Presentation Classification Error Rate
API	Application Programming Interface
AUC	Area Under the (ROC) Curve
BPCER	Bona fide Presentation Classification Error Rate
BPSK	Binary Phase-Shift Keying
CNN	Convolutional Neural Network
DFD	Data Flow Diagram
DSHOW	DirectShow (Windows camera capture backend)
EER	Equal Error Rate
FOURCC	Four-Character Code (video pixel-format identifier)
FPS	Frames Per Second
GUI	Graphical User Interface
IMU	Inertial Measurement Unit
JSON	JavaScript Object Notation
KYC	Know Your Customer
LBP	Local Binary Pattern
LFSR	Linear Feedback Shift Register
MJPG	Motion JPEG
OBS	Open Broadcaster Software
PAD	Presentation Attack Detection
ROC	Receiver Operating Characteristic
ROI	Region of Interest
rPPG	remote Photoplethysmography
SNR	Signal-to-Noise Ratio
UVC	USB Video Class
WB	White Balance
YUY2	Uncompressed packed YUV 4:2:2 pixel format

All abbreviations are listed in alphabetical order.

SYMBOLS AND NOTATIONS

$\mathbf{c_k}$	k-th chip of the challenge sequence, $c_k \in \{-1, +1\}$
n	Order (register length) of the LFSR
N	Period of the m-sequence, $N = 2^n - 1$
$\mathbf{f_c}$	Chip rate of the optical challenge (Hz)
$\mathbf{f_s}$	Measured camera frame rate (frames/s)
B, A	Emitter base luminance and modulation depth (grey levels)
s(t)	Emitted chip waveform (zero-order hold of c_k)
L(t)	Mean facial-ROI luminance measured by the camera
$\tilde{\mathbf{L}}\mathbf{(t)}$	Detrended facial luminance
τ	Hypothesised propagation / capture lag (ms)
ρ(τ)	Normalised (Pearson) correlation at lag τ
$\mathbf{S_{opt}}$	Optical liveness subscore, $\max_\tau \rho(\tau)$
$\mathbf{R_{ij}}$	Peak correlation of face ROI i against border zone j
$\mathbf{S_{sp}}$	Spatial assignment score
$\mathbf{f_0}$	Acoustic carrier frequency (4 kHz)
$\mathbf{z_k(\tau)}$	Complex baseband correlation of audio segment k at lag τ
m	Typing match fraction
$\mathbf{\Delta\bar{t}}$	Mean inter-key interval (s)
$\mathbf{\theta_{opt}, \theta_{typ}, \theta_{ac}}$	Per-lane pass thresholds (0.30, 0.70, 0.30)

ABSTRACT

Remote identity verification — video KYC, online examinations, telehealth consultations and executive video calls — now trusts a webcam to prove that a live human is present. That trust has been broken by real-time deepfakes and, more dangerously, by **video-injection attacks** that bypass the camera entirely through virtual-camera drivers. Because an injected stream never passes in front of the physical sensor, conventional passive presentation-attack detectors, which inspect texture or learned visual artefacts, are blind to it, and every improvement in generative models erodes them further.

This project presents **PRAESENS** (Latin, “being present”), a software-only liveness framework that turns ordinary laptop hardware into an **active physical challenge–response** system. For every session a fresh cryptographically-seeded maximum-length (m-)sequence is synthesised and broadcast through three physically independent channels: (i) the **optical lane** modulates the screen border so that the light reflected by the user’s skin carries the code, which is recovered from MediaPipe facial regions by detrending, lag-searched normalised cross-correlation and in-band SNR estimation; a **spatially-differential** extension drives left, right and top border zones with near-orthogonal shifted codes so that each facial region must answer its own zone; (ii) the **typing lane** verifies that a freshly generated phrase is typed with human keystroke timing while storing no characters; and (iii) the **acoustic lane** BPSK-modulates the same sequence onto a 4 kHz carrier and detects its room round-trip with a drift-tolerant segmented differential matched filter. A lag-plausibility gate and an explicit binary adjudicator fuse the lanes into an explainable ACCEPT / RE-CHALLENGE / REJECT verdict.

The system was implemented in about 11,700 lines of Python with 148 automated test cases and evaluated on 73 real sessions captured on commodity hardware. On a cross-session injection proxy of 3,448 trials, the optical lane alone achieves an **AUC of 0.938** and an **EER of 10.2 %**; bona fide sessions score a median of 0.73 against 0.055 for mismatched challenges, and the emitter-off control collapses to 0.009, proving that the score is caused by the emitted light and not by the presence of a face. All eight real virtual-camera injection sessions scored below the 0.30 pass threshold (maximum 0.18), and across thirteen fully-adjudicated live sessions **no attack was ever accepted**. A full three-lane bona fide session reached ACCEPT with optical 0.75, typing 1.00 and acoustic 0.93 (joint 0.89). Ablation confirms that removing either real lane withdraws 100 % of acceptances, while a spatial attack that defeats the global optical score (0.95) is still exposed by a spatial score of −0.02. PRAESENS requires no additional hardware, no training data and no cloud service, and its per-verification operating cost is effectively nil.

Keywords: liveness detection, deepfake, video injection attack, challenge–response, m-sequence, presentation attack detection, sensor fusion, keystroke dynamics, acoustic sensing.

CHAPTER 1

INTRODUCTION

1.1 Background

Over the last five years the webcam has quietly become a security device. Banks perform video-based Know-Your-Customer (KYC) onboarding, universities proctor examinations remotely, hospitals run telemedicine consultations, and companies authorise payments and hiring decisions over video calls. In every one of these workflows the relying party implicitly assumes that the pixels arriving from the camera were produced, at that moment, by light reflecting off a live human face that is physically in front of that camera.

That assumption no longer holds. Generative face-manipulation techniques — identity swapping, expression reenactment and fully synthetic faces — can now run in real time on consumer graphics hardware [1, 2, 3]. The consequences are no longer hypothetical: in early 2024 a finance employee of a multinational firm in Hong Kong transferred about US$25 million after a video conference in which every other participant, including the company's chief financial officer, was a deepfake [4]. Europol and ENISA both identify synthetic video and remote identity-proofing attacks as a growing, operational threat to law enforcement and to digital onboarding [5, 6].

The face anti-spoofing literature has historically focused on **presentation attacks** — printed photographs, screen replays or masks physically held in front of the sensor — standardised in ISO/IEC 30107-3 [7]. A far more scalable threat is the **video-injection attack**, in which the attacker never shows anything to a camera at all. A virtual-camera driver (for example OBS Virtual Camera) or a compromised capture pipeline feeds pre-recorded or live-generated frames directly to the application, which cannot tell them apart from a genuine webcam [6]. Injected frames contain none of the moiré, screen bezel, paper texture or depth cues that classical detectors depend on.

A fundamental observation motivates this project: *a genuine camera is coupled to the physical world, and an injected stream is not*. If the verifier changes the physical world in a way the attacker cannot predict — by emitting an unpredictable pattern of light, sound or interaction — then only a live, physically present user can produce a response that is correctly coded, correctly timed and correctly localised. PRAESENS is a software framework that implements this idea using nothing more than the screen, speaker, microphone, keyboard and webcam already present on every laptop.

1.2 Motivations

- **Passive detectors are in an arms race they are losing.** Classifiers trained to recognise artefacts of today's generators generalise poorly to unseen manipulation methods and

datasets [8, 9, 10]. A defence whose security rests on a physical law, rather than on a learned artefact, does not degrade when generators improve.

- **Injection attacks bypass presentation-attack detection by design.** Since the camera is never involved, any detector that analyses how a spoof looks to a camera is irrelevant. Verifying a *cause–effect relationship between a secret stimulus and the sensor reading* addresses injection directly.
- **Accessibility and cost.** Many published active-liveness schemes rely on depth sensors, infrared projectors or specialised smartphones. Students, small institutions and users in developing regions mostly use ordinary laptops. A solution that needs no extra hardware, no GPU, no training data and no cloud service can be deployed anywhere.
- **A bioinformatics perspective on the face as a biological signal source.** As students of Computer Science with a specialization in Bioinformatics, we approached the human face not as an image but as living tissue with measurable optical properties. Skin reflects incident light diffusely according to its melanin and haemoglobin content [11], and the same physics underlies remote photoplethysmography [12, 13]. Recovering a known, weak, pseudo-random code from a noisy biological measurement is conceptually the same matched-filter and sequence-correlation problem that pervades biological signal analysis.
- **Explainability and privacy.** Verification decisions affect people's access to banking, education and healthcare. We wanted every verdict to come with a human-readable reason and wanted the system to collect the minimum personal data possible — keystroke *timing* but never characters, numeric traces but never face images.

1.3 Scope of the Project

The scope of PRAESENS Project-I covers the complete design, implementation and empirical evaluation of a research-grade prototype:

- A per-session **challenge synthesiser** generating maximum-length LFSR sequences from a cryptographic seed, with derived spatial zone codes and an offline challenge phrase.
- A **screen emitter** (single-zone and three-zone spatial variants) with actual-redraw timestamp logging, and an **emitter-off** scientific control.
- Three independent evidence lanes — **optical**, **typing** and **acoustic** — plus an honest stub interface for an inertial lane on hardware that exposes an IMU.
- A **fusion and adjudication** layer producing ACCEPT / RE-CHALLENGE / REJECT verdicts with plain-language reasons.
- **Attack tooling** built against our own system: a virtual-camera injection harness and an adaptive injector that estimates the emitted pattern from the screen and modulates a source video in real time.
- An **evaluation pipeline** reporting ROC/AUC, EER, APCER, BPCER and ACER in the vocabulary of ISO/IEC 30107-3, per-condition robustness tables, ablation and cross-session analysis, together with a live **panel demonstration mode**.

Out of scope for this phase are mobile-phone deployment, large multi-subject demographic studies, and evaluation against commercial deepfake services; these are discussed as future work in Chapter 8.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

We organise prior work into five strands: the deepfake threat itself, passive texture- and learning-based anti-spoofing, physiological-signal methods, active challenge–response methods, and the signal-processing and fusion foundations on which PRAESENS builds. Fig. 2.1 summarises how the attack classes relate to the two families of defence.

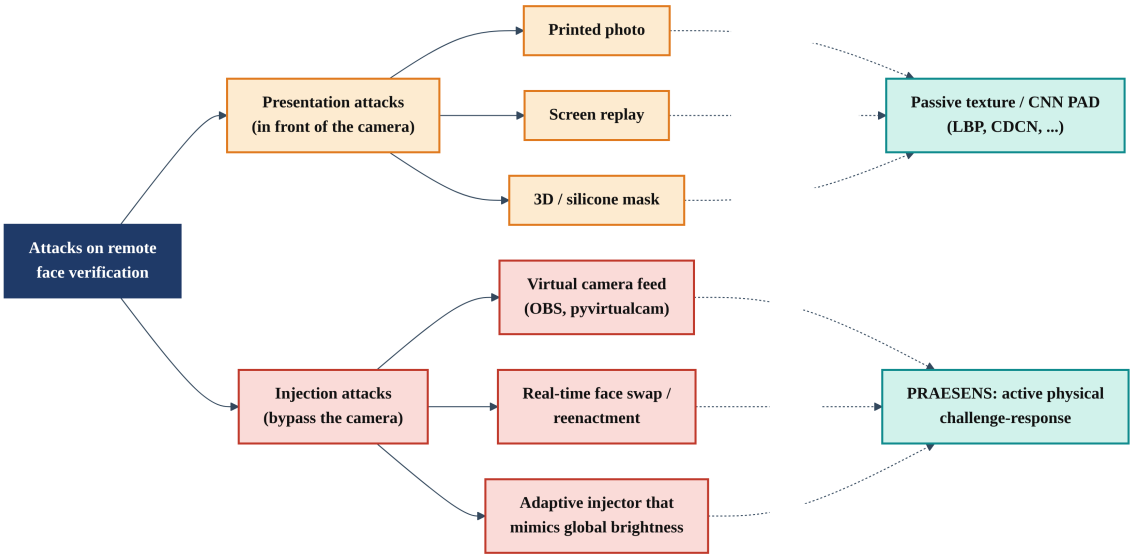


Fig. 2.1 Taxonomy of attacks on remote face verification and the defence family that addresses each

2.1.1 Face manipulation and the deepfake threat

Thies *et al.* demonstrated real-time facial reenactment of a monocular RGB target with Face2Face [1], showing that expression transfer could be performed live. Tolosana *et al.* [2] and Mirsky and Lee [3] survey identity swap, attribute manipulation and entire-face synthesis, and both conclude that detectors trained on one family of manipulation degrade sharply on others. Large benchmarks such as FaceForensics++ [8] and Celeb-DF [9] have driven progress in forensic classifiers, but Celeb-DF in particular showed that higher-quality fakes reduce the AUC of many published detectors to near chance.

2.1.2 Passive presentation-attack detection

Early anti-spoofing exploited micro-texture differences between real skin and recaptured media using Local Binary Patterns [14, 15], evaluated on datasets such as CASIA-FASD [16]. Boulkenafet *et al.* showed that colour-texture analysis in luminance–chrominance spaces generalises better than grey-scale LBP [17], and Tirunagari *et al.* used dynamic-mode decomposition to capture visual dynamics of replayed video [18]. Deep models followed: Liu *et al.* supervised CNN-RNN networks with auxiliary depth and rPPG cues [19], and Yu

et al. introduced central difference convolutional networks (CDCN) found through neural architecture search [20]. A recent survey [10] highlights that cross-dataset generalisation remains the central unsolved problem — and every one of these methods presumes that the spoof is *presented to* the camera.

2.1.3 Physiological-signal methods

Verkruysse *et al.* showed that the cardiac pulse is visible in ordinary video under ambient light [12]; Poh *et al.* recovered it with blind source separation [13], and de Haan and Jeanne [21] and Wang *et al.* [22] developed robust chrominance-based and plane-orthogonal-to-skin algorithms. Liu *et al.* used rPPG to detect 3D masks [23], and FakeCatcher [24] detects synthetic portrait video from the spatial-temporal coherence of biological signals. These methods are passive: a sufficiently good generator, or a replay of a genuine recording, carries a plausible pulse.

2.1.4 Active challenge–response liveness

Active methods change the environment and check the response. **Face Flashing** [25] flashes random colours on a phone screen and verifies the reflected light and its timing — the closest precursor to our optical lane. **FaceRevelio** [26] uses the phone screen as a controllable light source to recover a 3D surface normal map from a single front camera. On the acoustic side, **EchoPrint** [27] combines inaudible acoustic echoes with vision for two-factor face authentication, and **EchoFace** [28] detects media attacks from the acoustic reflection of a face. Keystroke dynamics have long been studied as a behavioural biometric [29, 30]. Each of these addresses a single channel, most are designed for smartphones, and none combines several physically independent channels under one shared secret.

2.1.5 Sequence design, detection and fusion foundations

Maximum-length sequences have a two-valued periodic autocorrelation — N at zero shift and −1 elsewhere — which makes them near-ideal probing signals [31, 32]. Their cross-correlation properties, and those of derived families such as Gold codes, are surveyed by Sarwate and Pursley [33, 34]. Matched filtering, BPSK modulation and coherent versus non-coherent detection are classical topics in digital communications [35]. On fusion, Kittler *et al.* [36] formalised classifier-combination rules and Ross and Jain [37] categorised fusion levels in multibiometric systems. ROC analysis [38] and the ISO/IEC 30107-3 error metrics [7] provide the evaluation vocabulary.

Table 2.1 Summary of representative literature and its limitations with respect to injection attacks

Work	Approach	Strength	Limitation for remote verification
Määttä *et al.* 2011 [14]	LBP micro-texture, SVM	Fast, simple	Needs a physical spoof medium; blind to injection
Boulkenafet *et al.* 2016 [17]	Colour texture (HSV / YCbCr LBP)	Better cross-dataset results	Learned artefacts; deepfakes have none of them

Liu *et al.* 2018 [19]	CNN-RNN with depth and rPPG supervision	Auxiliary physiological cues	Requires training data; replayable
Yu *et al.* 2020 [20]	Central difference CNN (CDCN)	State-of-the-art intra-dataset	Generalisation gap; GPU cost
Rössler *et al.* 2019 [8]	Forensic CNN on FaceForensics++	High accuracy on known fakes	Degrades on unseen generators [9]
Ciftci *et al.* 2020 [24]	Biological-signal coherence	Physiology-based	Passive; a replayed genuine video passes
Tang *et al.* 2018 [25]	Random screen colour flashes	Active, timing-verified	Single optical channel; smartphone-centred
Farrukh *et al.* 2020 [26]	Screen-lit photometric stereo	Recovers 3D shape	Single channel; tuned to phones
Zhou *et al.* 2018 [27]	Acoustic echo + vision	Two modalities	Phone earpiece geometry; trained model
Killourhy & Maxion 2009 [30]	Keystroke-dynamics anomaly detection	Behavioural biometric	Enrolment needed; stores typed text

2.2 Research Gap

The review exposes four gaps that PRAESENS is designed to close:

1. **No defence aimed squarely at injection on commodity laptops.** Passive PAD inspects appearance; injection attacks have perfect appearance. Active schemes exist, but almost all target smartphones with controlled screen–face geometry.
2. **Single-channel active methods are vulnerable to one well-engineered estimator.** An attacker who can observe the screen (a second camera, screen capture) can estimate a single global flash pattern and re-apply it to injected video. No prior system we found uses *spatially-differential* codes that force each facial region to answer its own part of the screen.
3. **No shared-secret fusion across physically independent channels.** Light, sound and keystrokes travel through entirely different physics. Binding all three to one per-session secret means an attacker must defeat every channel simultaneously, in real time, with correct timing.
4. **Weak treatment of measurement failure, explainability and privacy.** Most systems collapse “the sensor could not see the signal” and “this is an attack” into a single low score, produce an unexplained verdict, and retain biometric data. A trustworthy system must separate insufficient evidence from negative evidence, explain itself, and minimise what it stores.

2.3 Objectives

1. To design a per-session, cryptographically seeded challenge whose emitted signal has near-ideal autocorrelation and is fully reproducible from a stored seed for audit.

2. To recover that challenge from the light reflected by a user's face with an ordinary webcam, robustly to frame-rate jitter, auto-exposure, slow lighting drift and moderate variation in lighting, distance, eyewear and makeup.
3. To defeat the strongest single-channel counter-attack (global brightness re-modulation of injected video) through spatially-differential zone codes.
4. To add physically independent typing and acoustic lanes bound to the same session secret, under a strict keystroke-privacy contract.
5. To fuse the lanes with a lag-plausibility gate into an explainable three-way verdict that distinguishes REJECT from RE-CHALLENGE.
6. To evaluate the system honestly on real hardware using ISO/IEC 30107-3 metrics, scientific controls, ablation and real injection attempts, and to demonstrate it live.

2.4 Problem Statement

> *Given only the commodity sensors and actuators of a standard laptop — display, speaker, microphone, keyboard and webcam — design and evaluate a real-time verification system that decides whether the video stream presented to an application originates from a live human who is physically present in front of that device at that moment, such that pre-recorded, replayed, synthetically generated or virtually-injected video, including adaptive attacks that observe the emitted challenge, is rejected; such that every decision is explainable; and such that no face images or typed characters are retained.*

2.5 Project Plan

The project was executed as a sequence of fourteen measured milestones followed by hardening and evaluation phases. Each milestone ended with automated tests and, wherever possible, a real-hardware run whose numeric log was committed to the repository; a milestone notebook records what was actually measured rather than what was intended. Fig. 2.2 shows the schedule and Table 2.2 lists the milestones.

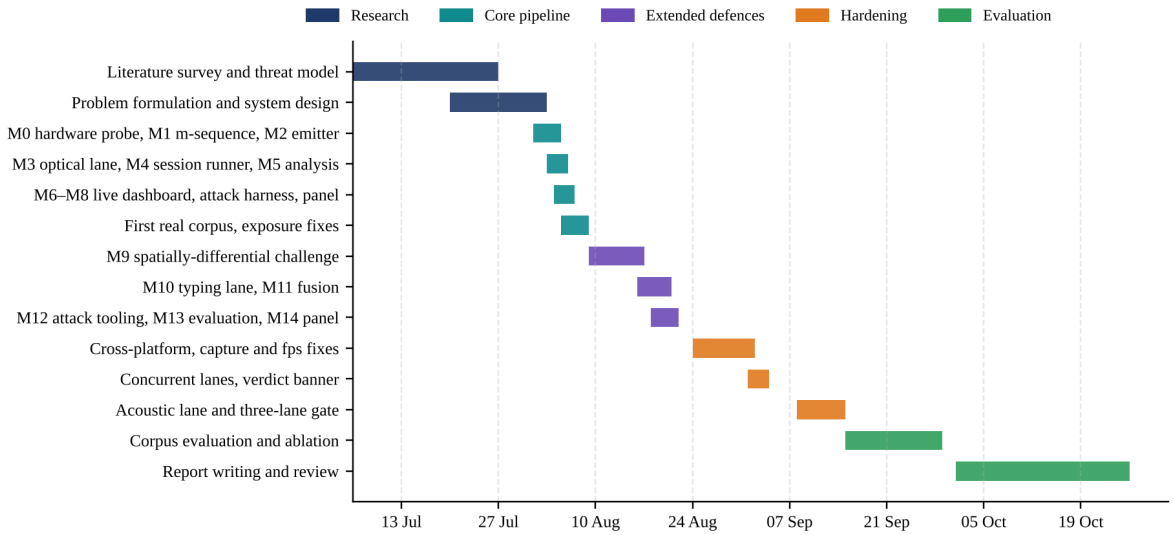


Fig. 2.2 Project plan (Gantt chart), July – October 2026

Table 2.2 Milestones and deliverables

No.	Milestone	Deliverable / acceptance evidence
M0	Hardware probe	Measured FPS, exposure/WB lock and display redraw capability
M1	Challenge synthesiser	LFSR m-sequence with verified maximal period and two-valued autocorrelation
M2	Emitter	Full-screen border emitter with actual-redraw `perf_counter` log; emitter-off control
M3	Optical lane	ROI luminance, detrending, lag-searched correlation, SNR gating, adaptive depth
M4–M5	Session runner, analysis	JSON session logs; ROC / EER / APCER / BPCER / ACER tables
M6–M8	Live dashboard, attack harness, panel	Rolling live score; instant camera-source switching; guided panel demo
M9	Spatially-differential challenge	3-zone emitter, ROI × zone matrix, spatial assignment score
M10	Typing lane	Privacy-preserving keystroke capture; phrase challenge
M11	Fusion and adjudication	Lag-gated fusion, three-way verdict with reasons, ablation harness
M12	Attack tooling	Adaptive screen-estimating injector, corpus plan, collapse-latency instrumentation
M13–M14	Evaluation extensions, panel demo	Per-attack metrics, robustness tables, scripted rehearsal mode
H1–H3	Hardening	Capture/FPS fixes, concurrent lanes, verdict banner, acoustic lane and three-lane gate

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional

Table 3.1 Functional requirements

ID	Requirement
FR1	Generate a fresh, unpredictable challenge (seed, LFSR order, chip rate, duration) for every session and persist the seed so the exact sequence can be regenerated for audit.
FR2	Render the challenge as screen-border luminance (single-zone or left/right/top zones) and log the actual time of every redraw.
FR3	Lock camera exposure and white balance, measure the steady-state frame rate, and choose a chip rate the camera can sample.
FR4	Locate the face, sample forehead and cheek luminance on every frame, and compute the optical subscore, lag and in-band SNR.
FR5	Compute the ROI × zone correlation matrix and spatial assignment score for spatial sessions.
FR6	Display a session-unique phrase and score keystroke correctness and timing without storing any character.
FR7	Play a BPSK acoustic probe and detect its round-trip lag and SNR through a single duplex audio stream.
FR8	Fuse lane results into ACCEPT / RE-CHALLENGE / REJECT with a plain-language reason; display a colour-coded banner and terminal summary.
FR9	Write one JSON log per session containing numeric traces, metadata and the full fusion record.
FR10	Provide attack tooling, a live panel demo, and an evaluation pipeline producing standard PAD metrics.

3.1.2 Non-Functional

Table 3.2 Non-functional requirements

ID	Attribute	Requirement
NFR1	Security	Challenge seed drawn from the OS CSPRNG (`secrets`); nothing an attacker can observe in advance predicts the sequence.
NFR2	Privacy	No face image, video or typed character is ever stored; logs hold numeric traces only; attack videos stay in a git-ignored folder.
NFR3	Real-time performance	Single shared capture loop; face landmarks every 2nd frame on a down-scaled copy; live verdict within one session (≈ 20–40 s).
NFR4	Robustness	Never crash on a missing device; degrade to *no_evidence* with a reason instead of fabricating a score.

NFR5	Explainability	Every verdict carries a sentence naming the deciding lane, its subscore and its lag.
NFR6	Portability	Runs on Windows and macOS; camera backend chosen per OS; CPU-only inference.
NFR7	Configurability & reproducibility	Every tunable lives in `config.yaml`; every session re-scorable offline from its log.
NFR8	Maintainability	Modular lanes with a uniform `LaneResult` interface; automated tests for every module.

3.2 Feasibility Study

3.2.1 Technical Feasibility

Technical feasibility was established empirically before any lane was built (Milestone 0). The hardware probe confirmed that a consumer UVC webcam can have its exposure and white balance locked, that the display can redraw a full-screen border far faster than the chip rate, and that a ±110 grey-level border modulation produces a luminance change on facial skin that is measurable above sensor noise. The principal technical risk discovered was that the frame rate achievable at a face-detectable exposure was as low as 8 fps; this was addressed by adaptive chip-rate selection (Section 5.1.3). Mature open-source libraries — OpenCV [39], MediaPipe face and hand landmarkers [40, 41, 42], NumPy [43] and SciPy [44] — cover all perception and signal-processing primitives on CPU.

3.2.2 Economic Feasibility

PRAESENS requires no hardware beyond a standard laptop and uses only free and open-source software. There is no model-training cost, no labelled dataset to purchase and no per-call cloud fee. A detailed cost analysis is presented in Section 7.6; the marginal operating cost of a verification is a fraction of a paisa in electricity.

3.2.3 Social Feasibility

The system protects people from impersonation fraud while being respectful of them: the light pattern is a gentle grey-level border change, the acoustic probe is a short tone, and the typing challenge is five ordinary words. The keystroke-privacy contract (only timing and a per-key boolean are retained) and the no-images policy reduce the data-protection burden for institutions adopting the system. Because the method is training-free, it does not inherit demographic bias from a training set; robustness across skin tone is nevertheless treated as an explicit evaluation axis, and low-reflectance conditions are routed to RE-CHALLENGE rather than REJECT.

3.3 System Specification

3.3.1 Hardware Specification

Table 3.3 Hardware specification (development and evaluation platform)

Component	Specification used	Minimum recommended
Computer	ASUS TUF A16 laptop (Windows); MacBook for macOS compatibility testing	Any dual-core x86/ARM laptop
Camera	Built-in / USB UVC webcam, 640 × 480, exposure-locked (8–30 fps depending on exposure)	UVC webcam supporting manual exposure, ≥ 15 fps
Display	1920 × 1080 laptop panel, full-screen emitter window	Any display ≥ 60 Hz refresh
Audio output	Realtek built-in speakers (MME)	Built-in speakers (not Bluetooth earbuds)
Audio input	Realtek microphone array (MME), 44.1 kHz	Built-in microphone, 44.1 kHz
Keyboard	Built-in keyboard	Any keyboard
Memory / storage	Standard laptop memory; < 50 MB disk for code, models and logs	4 GB RAM

3.3.2 Software Specification

Table 3.4 Software specification

Software	Purpose
Python 3.11+	Implementation language
OpenCV (opencv-python)	Camera capture, exposure locking, full-screen rendering, banner GUI
MediaPipe Tasks (FaceLandmarker, HandLandmarker)	Facial landmarks for ROI placement; hand tracking for the standalone typing mode
NumPy, SciPy	LFSR, correlation, Butterworth filtering, baseband demodulation
sounddevice (PortAudio)	Duplex audio playback and capture for the acoustic lane
pynput	OS keyboard hook for keystroke timing
mss, pyvirtualcam, OBS Studio	Attack tooling: screen capture and virtual-camera injection
Matplotlib, PyYAML	Evaluation figures; configuration
pytest	Automated testing (148 test cases)
Git / GitHub	Version control; session logs committed as evidence
Windows 11, macOS	Target operating systems

CHAPTER 4

DESIGN APPROACH AND DETAILS

4.1 System Architecture

PRAESENS is organised as five layers (Fig. 4.1). The **challenge layer** draws one secret per session and derives every channel's stimulus from it. The **emission layer** uses the host's own screen and speaker as actuators. The **sensing layer** observes the physical response through one shared camera capture loop, one duplex audio stream and one keyboard hook — never two competing consumers of the same device. The **evidence lanes** turn raw sensor data into a uniform `LaneResult` (subscore, status, lag, confidence, diagnostics). Finally the **fusion and adjudication** layer applies lag-plausibility and binary gating rules to reach a verdict, which is shown to the user and written to a JSON log.

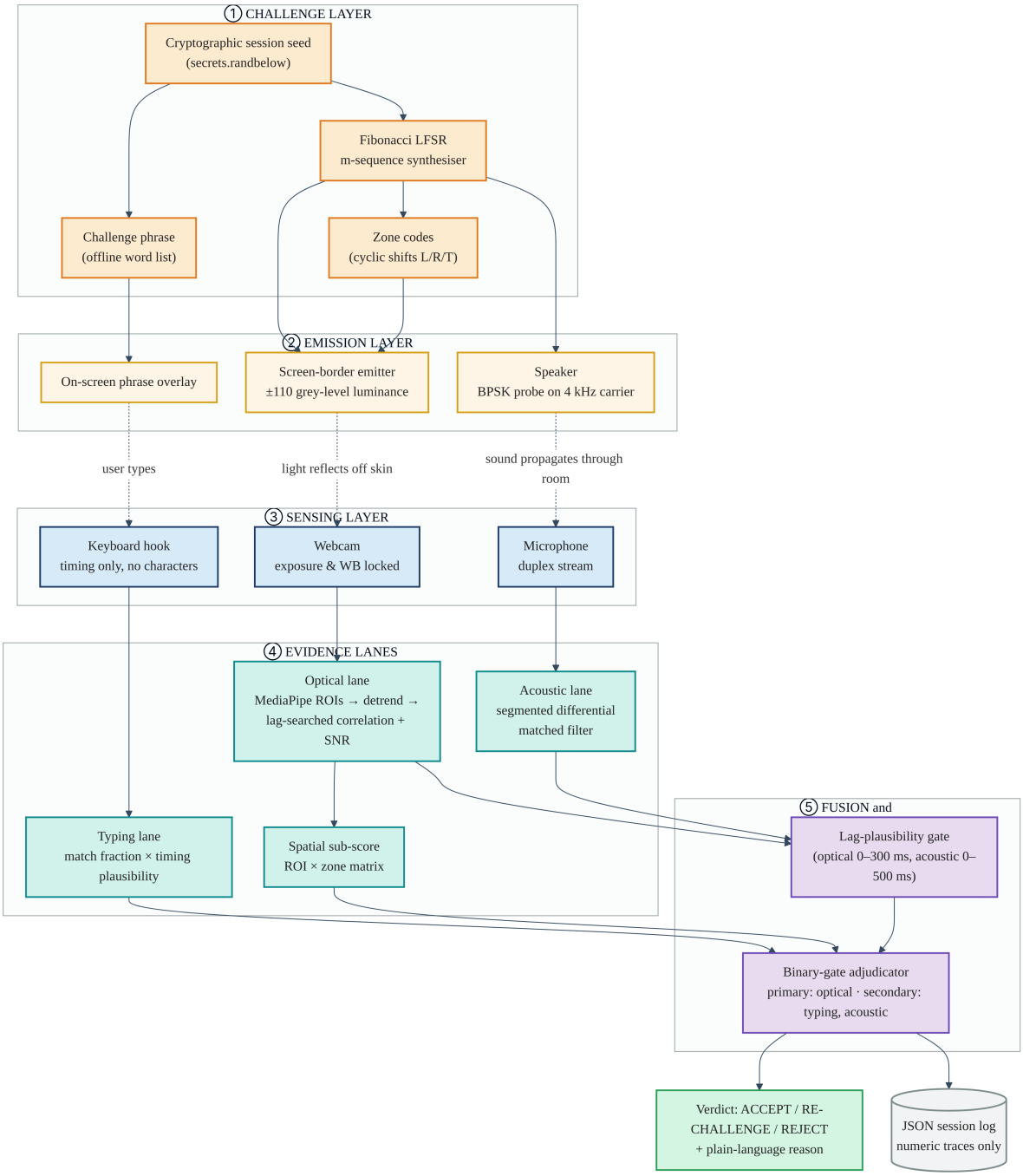


Fig. 4.1 Layered system architecture of PRAESENS

Three design principles run through the architecture:

- **One secret, many physics.** Light, sound and keystrokes propagate through unrelated physical media, but all carry functions of the same seed, so a single saved seed regenerates every lane's expected signal exactly.
- **Joint temporal coherence, not an averaged score.** A lane counts only if its response arrives at a physically plausible lag (0–300 ms for camera latency, 0–500 ms for an audio round trip). A high correlation at an implausible lag — a lucky coincidence or an attacker who matched magnitude but not timing — contributes nothing.

- **Honest degradation.** A lane that cannot measure says *no_evidence* or *insufficient_signal*; it never fabricates a zero. This keeps "we could not see" distinct from "we saw an attack".

Fig. 4.2 shows actual frames produced by the spatial emitter's renderer: the left, right and top border zones carry independent codes while the centre shows the live camera preview.



Fig. 4.2 Four successive frames rendered by `SpatialEmitter.render_frame()`; each border zone switches independently between B+A and B−A

4.2 Design

4.2.1 Data Flow Diagram

The context-level (Level 0) DFD in Fig. 4.3 treats PRAESENS as a single process exchanging data with the claimant, the relying party and the operator. The Level 1 DFD in Fig. 4.4 decomposes it into seven processes and three data stores. Note the data store D2: the optical lane correlates against the emitter's *logged actual redraws*, not an idealised schedule, because operating-system scheduling jitter would otherwise blur the correlation peak.

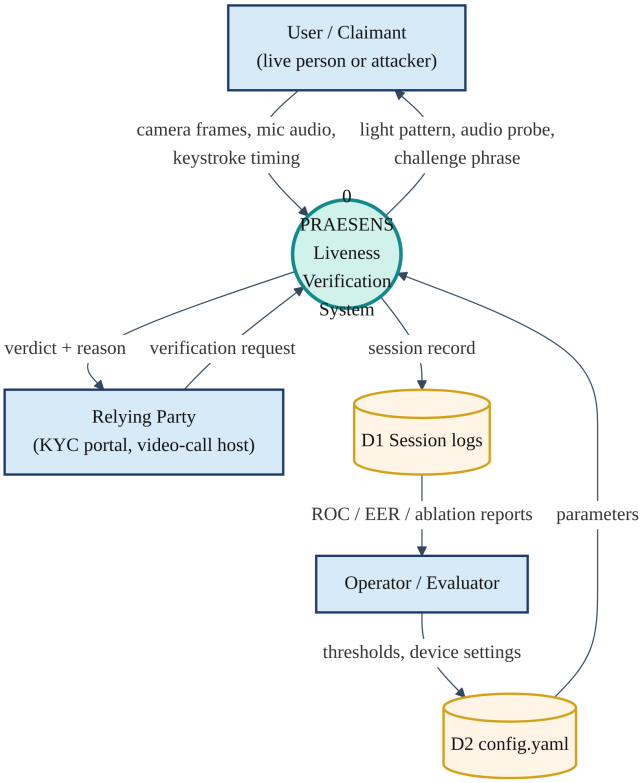


Fig. 4.3 Level 0 (context) data flow diagram

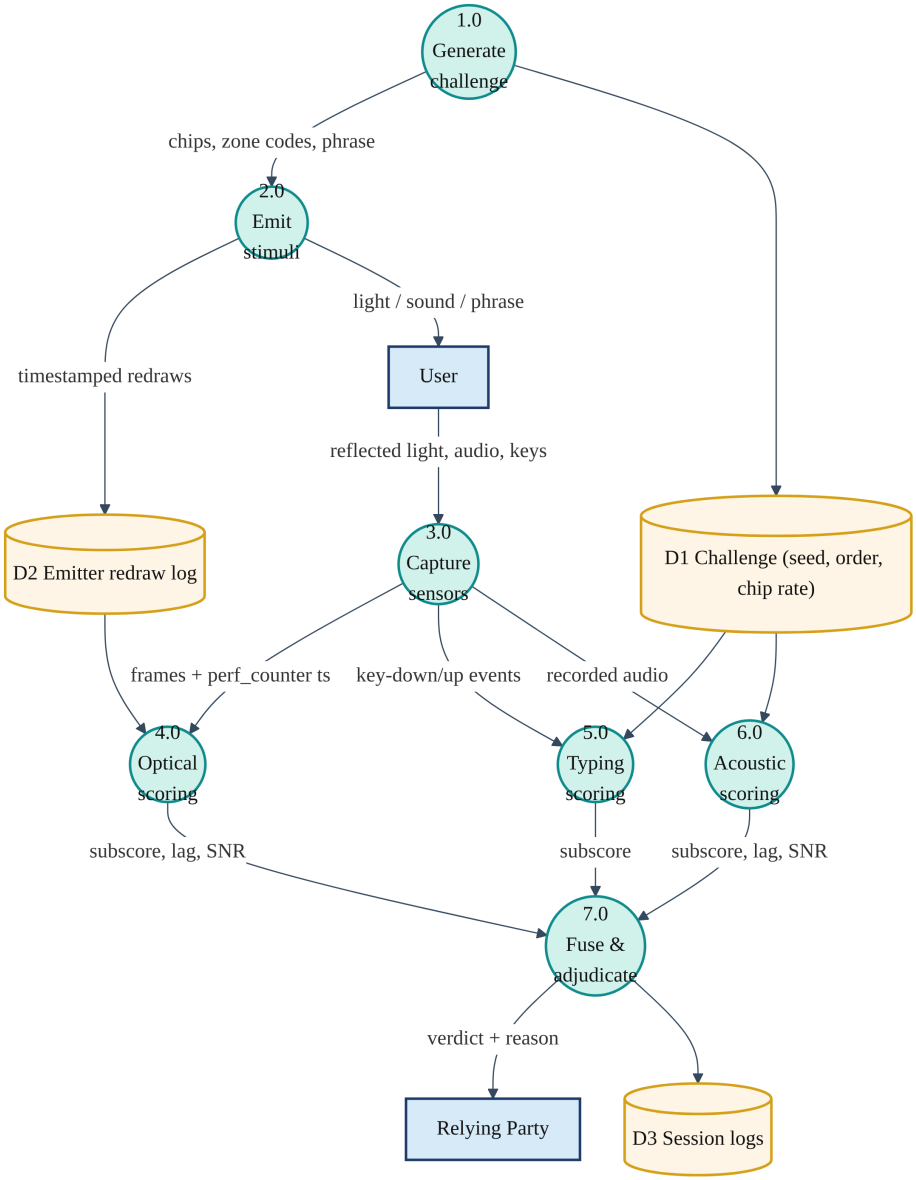


Fig. 4.4 Level 1 data flow diagram

4.2.2 Use Case Diagram

Four actors interact with the system (Fig. 4.5): the **claimant**, who starts a session and responds to the light, typing and acoustic challenges; the **attacker**, whose injected or replayed video must be caught by the light-challenge use case; the **operator / researcher**, who configures thresholds, runs the panel demonstration and collects evaluation corpora; and the **relying party**, which consumes the verdict and its reason.

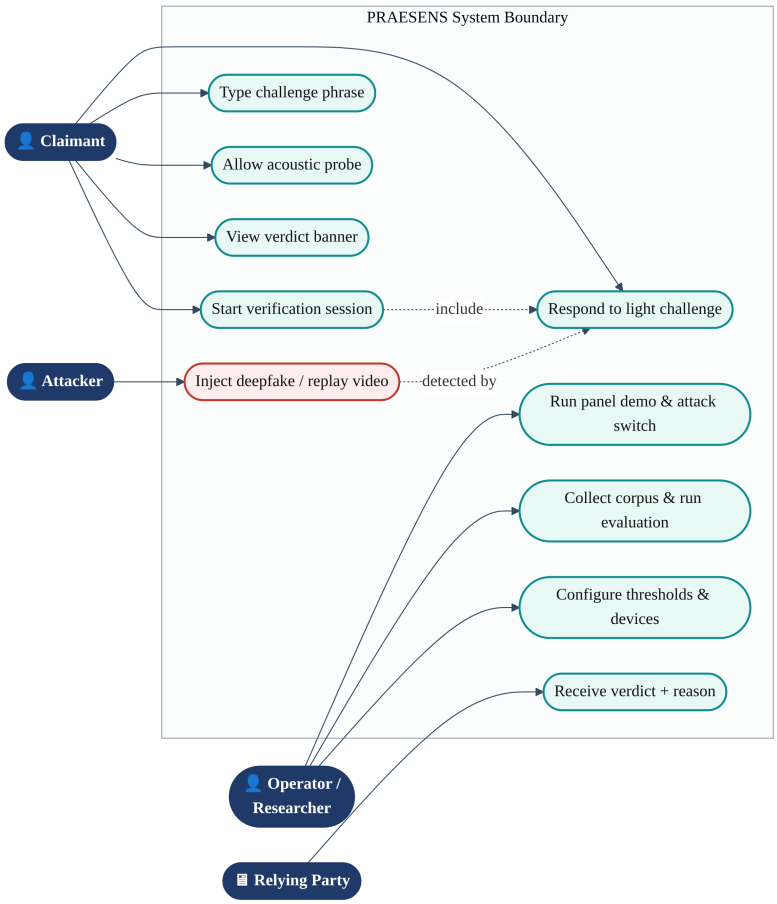


Fig. 4.5 Use case diagram

4.2.3 Class Diagram

Fig. 4.6 shows the principal classes. `Challenge` is an immutable value object that composes three `ZoneChallenge`s. Both emitters render a challenge and keep a redraw log. Each lane's processor produces a lane-specific result (`OpticalResult`, `AcousticResult`, keystroke events) that is wrapped into the uniform `LaneResult`, which is the only type the `Adjudicator` and `adjudicate_two_lane()` consume. This uniform interface is what allowed new lanes to be added without changes to fusion.

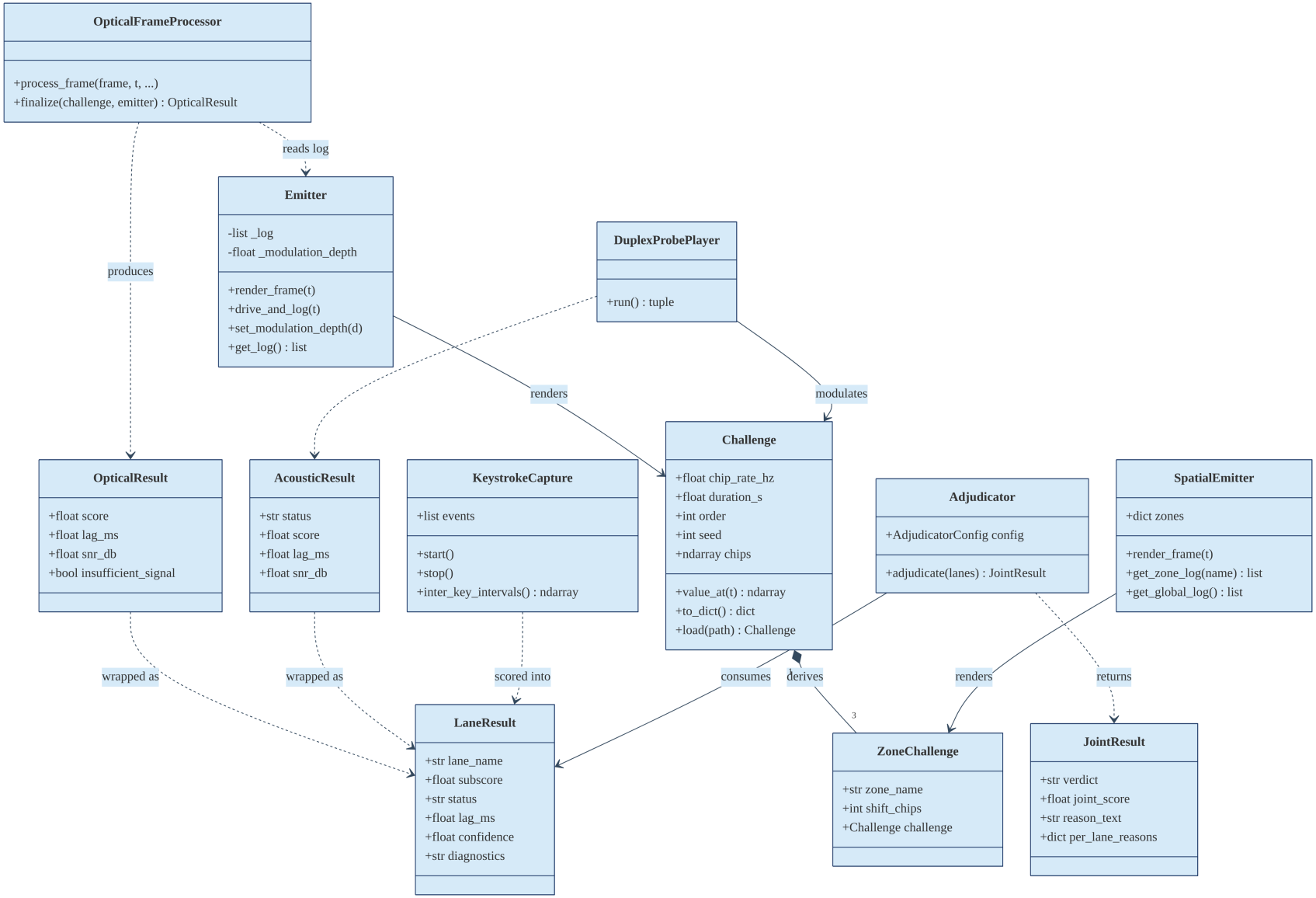


Fig. 4.6 Class diagram of the core `praesens` package

4.2.4 Sequence Diagram

Fig. 4.7 traces one three-lane session. After exposure is locked and the steady-state frame rate measured, the chip rate is chosen; then the screen pattern, the acoustic probe and the typing phrase all run *concurrently* in one challenge window. Each camera frame is grabbed once and handed to the optical processor. Only after the window closes are the lanes finalised and adjudicated, so no lane can influence another's measurement.

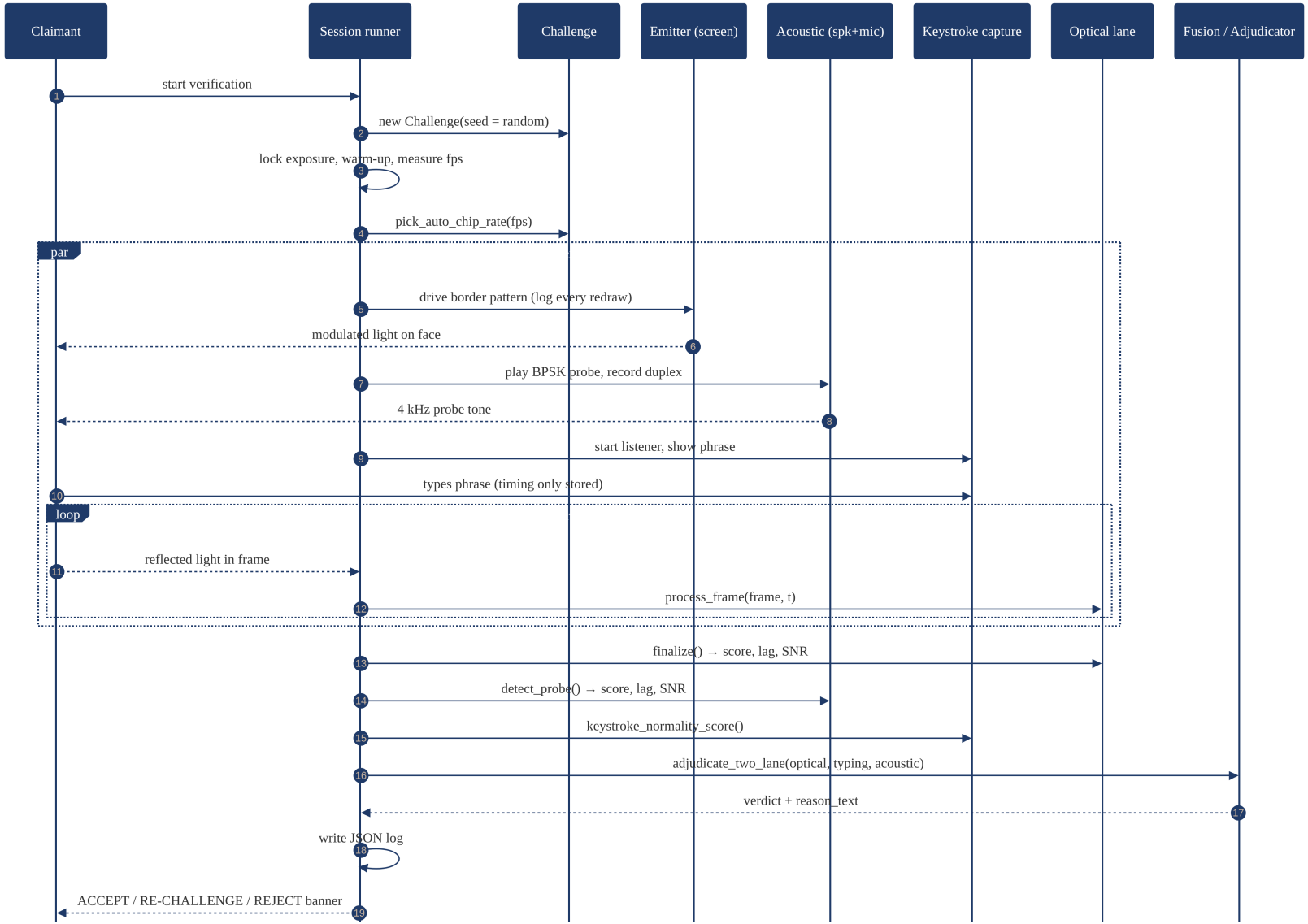


Fig. 4.7 Sequence diagram of a concurrent three-lane verification session

4.2.5 Verdict Decision Logic

The final adjudication policy is an explicit truth table (Fig. 4.8): the optical lane is *primary* — if it fails, no other lane can rescue the session (REJECT). Typing and acoustic are *secondary* — if optical passes but a secondary lane fails, the user is asked to retry (RE-CHALLENGE) rather than being rejected outright. ACCEPT requires every lane that ran to pass.

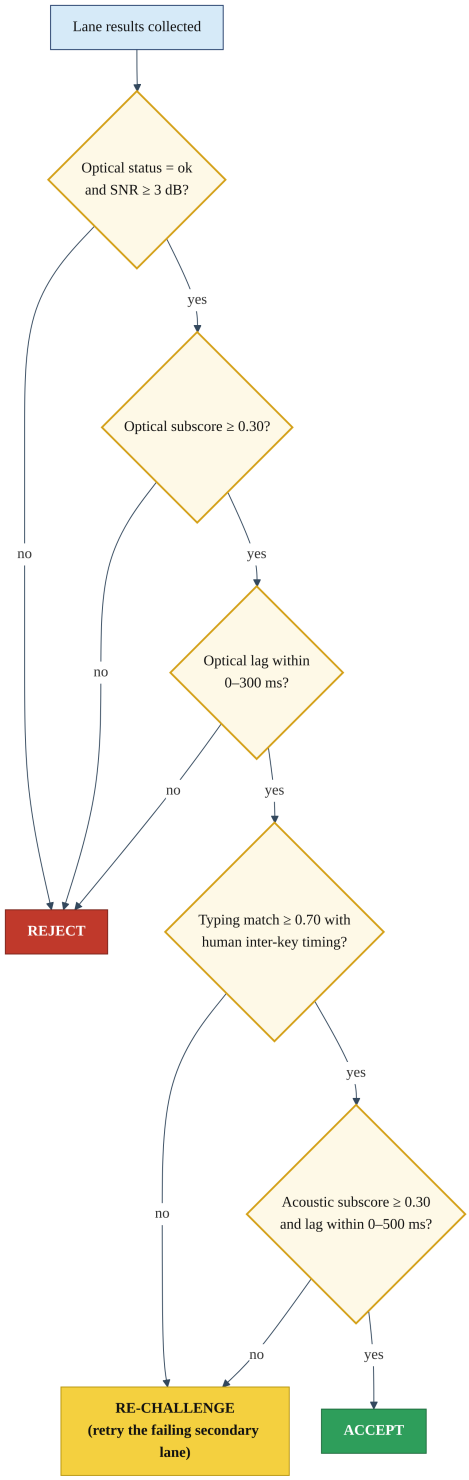


Fig. 4.8 Binary-gate adjudication flowchart

CHAPTER 5

METHODOLOGY AND TESTING

This chapter describes each module of PRAESENS, the mathematics it implements and the design decisions behind it, followed by the testing strategy. The code base comprises twelve core modules in the `praesens` package (≈ 4,600 lines) plus demonstration, attack, evaluation and diagnostic tooling, for a total of roughly 11,700 lines of Python.

5.1 Module Description

5.1.1 Challenge Synthesiser (`challenge.py`)

The challenge is a ±1 maximum-length sequence produced by a Fibonacci linear feedback shift register of order n. With a primitive feedback polynomial the register visits all 2^n-1 non-zero states before repeating, and the resulting sequence has the two-valued periodic autocorrelation [31, 32]

$$R(k) = (1/N)\,\Sigma_i\, c_i\, c_{i+k} = 1 \text{ for } k \equiv 0 \pmod N,\quad -1/N \text{ otherwise},\quad N = 2^n - 1 \tag{5.1}$$

This impulse-like property means a genuine reflection yields one sharp correlation peak, and periodic interference such as 50 Hz mains flicker cannot accidentally align with the pattern. Rather than trusting a published tap table — whose conventions differ between sources and silently produce short cycles if mismatched — the module *searches* for a tap set whose simulated period equals 2^n-1 under its own shift convention. The smallest order covering the required number of chips is chosen, and the seed (the initial register state) is drawn from the operating system's cryptographic random source. Because (order, seed, chip rate, duration) fully determine the sequence, a session log needs to store only those four numbers to be independently re-scorable. Fig. 5.1 shows a 63-chip sequence, its autocorrelation and the near-zero mutual correlation of the three spatial zone codes.

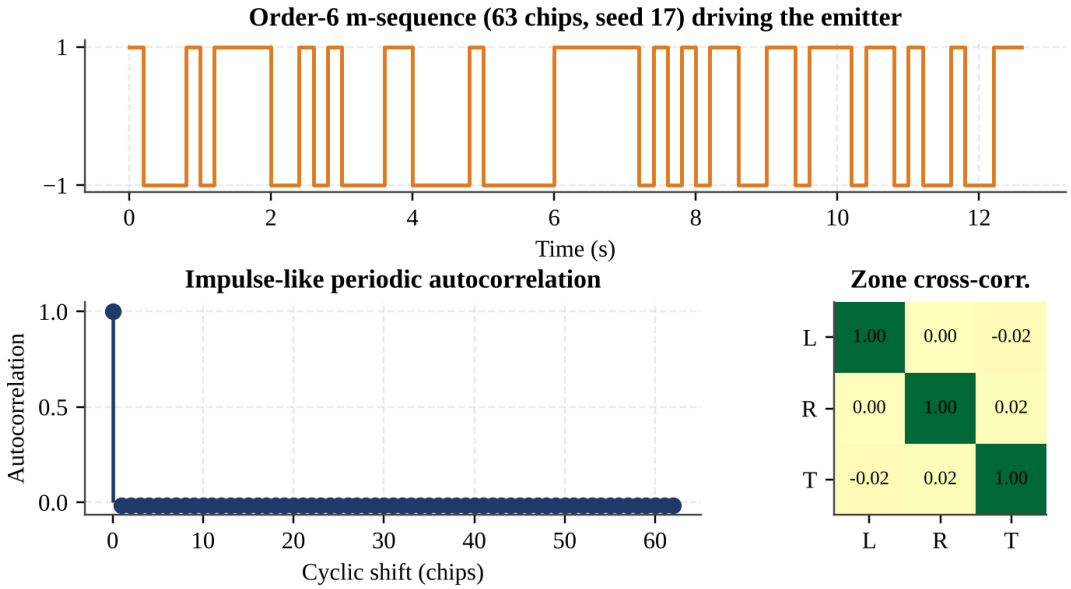


Fig. 5.1 Order-6 m-sequence, its ideal periodic autocorrelation, and the cross-correlation of the three derived zone codes (diagonal 1.00, off-diagonal ≤ 0.02)

Zone codes. For the spatial challenge the three zones receive cyclic shifts of the same full-period sequence, spaced N/3 chips apart. A cross-correlation between shifted copies is the autocorrelation at that shift, so the zones are guaranteed near-orthogonal by Eq. (5.1). Shifting is done on the *full* period before truncation, since rolling a truncated excerpt would break the property.

Challenge phrase. For the typing lane, five words are recombined from a hand-authored list of about 150 common words, seeded deterministically from the session seed. A `generative` flag that would call a language model is hard-pinned off and raises an error if enabled, so the demonstration path never depends on a network service.

5.1.2 Emitter (`emit.py`, `spatial.py`)

The emitter renders a full-screen window whose border (25 % of each edge in sessions; 14 % in the panel demo) takes luminance

$$\ell(t) = clip(\, B + A \cdot s(t),\, 0,\, 255\,), \quad B = 127, A = 110 \tag{5.2}$$

where s(t) is the zero-order-hold chip value. The modulation depth A was raised from 60 to 110 after a real gate run scored a bona fide session 0.692, just short of the original 0.70 bar, while the emitter-off control stayed at 0.000 — the control behaving correctly showed that strengthening the stimulus, not lowering the bar, was the right fix. Every redraw records `time.perf_counter()` at the moment the frame is actually pushed to the window, so the optical lane aligns against what was truly on screen. The **emitter-off** condition renders a constant B: since the chips are zero-mean, this has exactly the same average luminance, isolating the pattern as the only changed variable. The `SpatialEmitter` paints left, right and top zones with independent zone codes and exposes the same `drive_and_log()` interface, so demonstrations can switch between the two emitters without changing their run loop.

5.1.3 Optical Lane (`optical.py`, `capture.py`)

The optical lane is the primary evidence lane. Its processing chain is:

1. **Capture with honest timestamps.** Each frame is timestamped at `grab()`, before decoding, so the time reflects when light reached the sensor. A single shared capture helper negotiates pixel format, resolution and frame rate in a DirectShow-safe order.
2. **Exposure and white-balance lock.** Auto-exposure would actively cancel the brightness changes being measured. The camera is locked to a manual exposure value, verified by checking luminance stability over 15 frames, and then warmed up for 30 frames before the frame rate is measured.
3. **Adaptive chip rate.** The chip rate is set to $f_c = \mathrm{clip}(f_s/10, 0.5, 5)$ Hz and the session is lengthened to emit at least 60 chips, so the sequence is always sampled ≥ 10 times per chip regardless of what frame rate the room's lighting permits.
4. **Facial ROIs.** MediaPipe's FaceLandmarker [41] locates 478 landmarks; boxes over the forehead and both cheeks — flat, largely shadow-free skin — are shrunk by 6 % of face height to stay clear of eyes, brows, nose and lips. Landmarks are computed every second frame on a 480-pixel-wide copy, but ROI luminance is sampled on *every* frame.
5. **Detrending.** A centred, time-windowed (1.5 s) moving average is subtracted to remove slow lighting drift: $\tilde{L}(t) = L(t) - \mathrm{mean}_{|u-t| \le 0.75\ \mathrm{s}}\, L(u)$.
6. **Lag-searched normalised correlation.** Because frames are not evenly spaced, the emitted log is re-sampled at $(t_i - \tau)$ for every lag hypothesis rather than shifting sample indices:

$$\rho(\tau) = \Sigma_i\, (\tilde{L}_i - \mu_L)(s(t_i - \tau) - \mu_s) \,/\, (N\, \sigma_L\, \sigma_s), \quad S_{opt} = max_{0 \le \tau \le 300\ ms}\, \rho(\tau) \qquad (5.3)$$

The lag grid is 0–300 ms in 5 ms steps; only non-negative lags are physical, since the reflection cannot precede the stimulus.

1. **SNR gating.** A random chip sequence has a sinc^2-shaped spectrum with its main lobe from DC to f_c, so signal power is measured in the band [0.3 Hz, f_c] with a 4th-order Butterworth filter on a 60 Hz uniform resampling, against the residual out-of-band power: $\mathrm{SNR} = 10 \log_{10}(P_{in}/P_{out})$. Below 3 dB the session is marked *insufficient_signal*, which routes to RE-CHALLENGE/no-evidence rather than being counted as an attack. If the SNR is low in the first 3 s, the modulation depth is boosted adaptively.
2. **Confidence.** The lane confidence grows with the SNR margin above the floor, clipped to [0.05, 1.0].

A synthetic-signal test early in development returned an impossible negative SNR for a strong, clean signal; it exposed that a band centred *on* the chip rate captures only a sliver of an m-sequence's power. The band definition above is the corrected one.

5.1.4 Spatially-Differential Challenge (`spatial.py`)

A single global code has a known weakness: an attacker who can see the victim's screen can estimate the one emitted waveform and multiply the brightness of the injected video by it, and the global score would pass. With three zones driven by near-orthogonal codes, a genuine face is lit predominantly by its nearest zone — the forehead by the top strip and each cheek by its side strip — so the ROI × zone peak-correlation matrix R must be diagonal-dominant. The spatial assignment score is

$$S_{sp} = mean_i\, R_{i,\,z(i)} - mean_i\, max_{j \neq z(i)}\, R_{ij} \tag{5.4}$$

where z(i) is the expected zone of ROI i. A global multiply attack has only one waveform, so every ROI correlates almost equally with every zone and S_{sp} collapses to zero (Fig. 5.2). Because the camera and screen face the subject from the same side, no left–right flip is needed between the emitter's frame and the camera's frame.

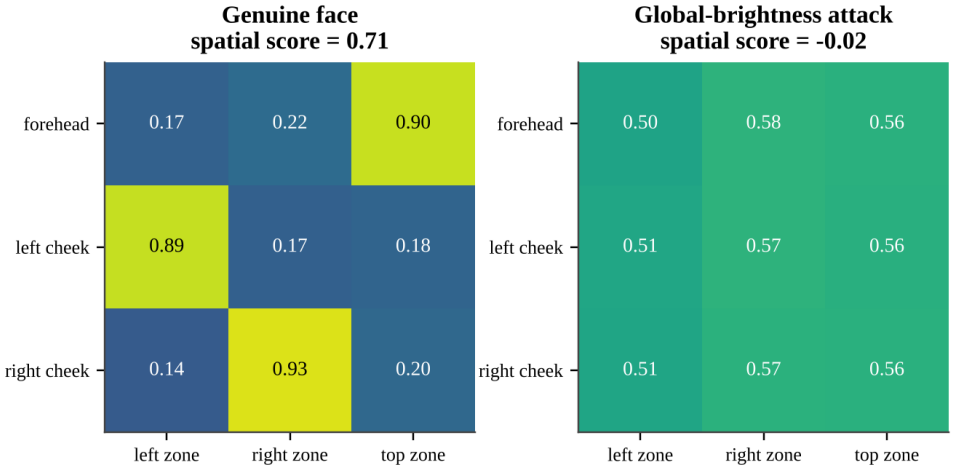


Fig. 5.2 ROI × zone correlation matrices computed by `compute_correlation_matrix()`: a genuine face is diagonal-dominant (S_{sp} = 0.71) while a global-brightness attack is uniform (S_{sp} = −0.02)

5.1.5 Typing Lane (`typing.py`)

The typing lane checks live engagement with a challenge that could not have been pre-recorded. A `KeystrokeCapture` listener compares each key press with the next expected character of the session phrase and stores **only** the key-down time, key-up time and a boolean `matched_expected`; the character itself exists only as a local variable during one comparison. A structural test types the literal word "secret" and asserts that every stored event has exactly those three keys and no string value anywhere. The score is

$$S_{typ} = m \quad if\ 0.03\ s \le \Delta\bar{t} \le 3.0\ s, \quad else\ 0.3{\cdot}m, \quad m = n_{matched} / n_{expected} \tag{5.5}$$

where $\Delta\bar{t}$ is the mean inter-key interval. The penalty catches naive scripted keystroke injection, which types with unnaturally fast, uniform timing. Fewer than three keystrokes gives *no_evidence*. The lane passes when $S_{typ} \ge 0.70$. An earlier design (Milestone 10) also correlated keystroke impulses with MediaPipe hand motion [42] over a ±300 ms window; it

remains available standalone, but was removed from the verdict after real sessions showed that a webcam framed on the face cannot see the hands while typing.

5.1.6 Acoustic Lane (`acoustic.py`)

The acoustic lane BPSK-modulates the *same* session chip sequence onto a 4 kHz carrier (44.1 kHz sampling, amplitude 0.6) with 20 ms raised-cosine chip transitions, plays it through the speaker and records the microphone on one duplex stream. Detection must survive two real-world effects discovered on hardware: speaker/microphone clock drift and dropped audio buffers, which cancelled a single 20-second coherent correlation to nearly zero. The detector therefore demodulates to complex baseband (10 Hz low-pass), splits the reference into 1 s segments and forms a *differential* statistic

$$D(\tau) = \sqrt{\left(\left|\Sigma_k \, z_k(\tau)\, z_{k-1}^{*}(\tau)\right| \,/\, \Sigma_k \, \|z_k\| \, \|z_{k-1}\| \right)} \tag{5.6}$$

in which a constant clock-drift phase step between segments cancels in the magnitude, while at a wrong lag the random chip signs make the products cancel. The noise floor is the larger of (a) the same statistic computed at off-carrier frequencies (±300, ±600 Hz) and (b) the statistic for three decoy codes — shifted copies of the session sequence — so that a probe heard at the wrong alignment cannot pass on a sidelobe. The lag search covers 0–500 ms at 1 ms resolution, and a peak on the search edge is rejected. Fig. 5.3 illustrates the probe and detector output.

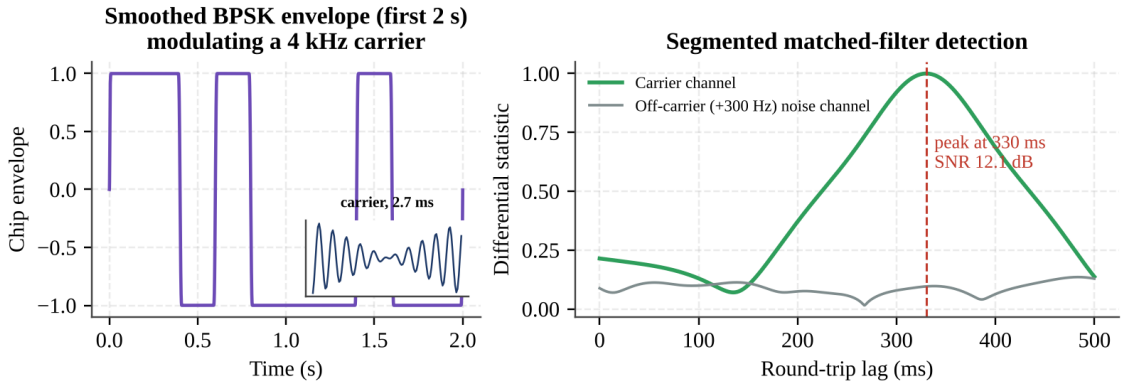


Fig. 5.3 Acoustic probe envelope with a 4 kHz carrier inset, and the segmented differential matched-filter statistic for a simulated 330 ms round trip versus an off-carrier noise channel

Hardware debugging also uncovered a practical failure mode worth reporting: on the test laptop the default output was a pair of Bluetooth earbuds and the speakers were muted, so the microphone heard no tone at all. The lane now supports named input/output devices, warns when the output looks like headphones, and reports “tone not heard” as an actionable reason.

5.1.7 Fusion and Adjudication (`fusion.py`)

Every lane emits a `LaneResult`. A lane *contributes* only when its status is *ok* and its lag lies within its physical window (optical 0–300 ms, acoustic up to 500 ms). Two policies are implemented:

- **Confidence-weighted Adjudicator** (used for ablation): joint score = $\Sigma\ w_l\ S_l\ /\ \Sigma\ w_l$ over contributing lanes, ACCEPT ≥ 0.60, REJECT < 0.30, RE-CHALLENGE in between, with a 0.05 hysteresis margin for the live display. A safeguard downgrades ACCEPT to REJECT if any contributing lane's own subscore is below 0.30 — added after a test showed a confident typing lane could numerically out-vote an optical lane that already looked like an attack.
- **Binary-gate policy** `adjudicate_two_lane()` (used for live sessions, Fig. 4.8): optical must pass on both score and lag; secondary lanes failing yields RE-CHALLENGE; optical failing yields REJECT.

Each verdict carries a sentence such as "all 3 lanes agree: optical 0.75, typing 1.00, acoustic 0.93" or "optical lane failed — scored 0.17, below pass threshold", and the same helper drives both the terminal block and the on-screen banner, so they can never disagree.

5.1.8 Attack Tooling (`demo/attack.py`, `attacks/adaptive_injector.py`)

A defence is only as credible as the attacks it has been tested against, so we built attacks against our own system. The **attack harness** opens every camera source (real webcam, OBS Virtual Camera) at start-up and switches between them with a pointer swap, so the audience sees the rolling score collapse without a reset. The **adaptive injector** is the strongest attack we could construct: it screen-captures the emitter's border regions, estimates the emitted waveform purely from pixel brightness (exactly what a second camera pointed at the monitor would see — it is never given the seed), and multiplies the luminance (Y channel) of a source video by that estimate with a configurable processing lag, outputting through a virtual camera. Global and per-zone modes are supported.

5.1.9 Evaluation Pipeline (`eval/`)

`analyse.py` computes the score distribution per condition, ROC/AUC, EER and APCER/BPCER/ACER at the EER threshold [7, 38], per-attack-type metrics at the pooled threshold, per-condition robustness tables and collapse-latency statistics. Emitter-off sessions are reported as a control and insufficient-signal sessions as measurement failures; neither is folded into the attack class. `cross_session.py` builds an injection proxy without attack recordings: every genuine trace is scored against every *other* session's challenge — physically what an injected video looks like to the verifier, a real face responding to a pattern it never saw. `ablate.py` recomputes verdicts with each lane held out.

5.2 Testing

5.2.1 Testing Strategy

Testing followed four levels:

1. **Unit tests on synthetic signals.** Deterministic, hardware-free tests for every module (pytest), each tied to a specific claim — for example, that a high subscore at an implausible lag contributes nothing, or that no character is ever stored.
2. **Mocked-hardware integration tests.** Camera, audio and keyboard are mocked to verify orchestration properties, such as exactly one `open_camera()` call and both lanes receiving the literal same frame object.
3. **Real-hardware sessions.** Every milestone ended with a real run whose JSON log was committed, including deliberate missing-device runs to verify graceful degradation.
4. **Scientific controls and adversarial testing.** Emitter-off controls, virtual-camera injection, the adaptive injector and cross-session mismatching.

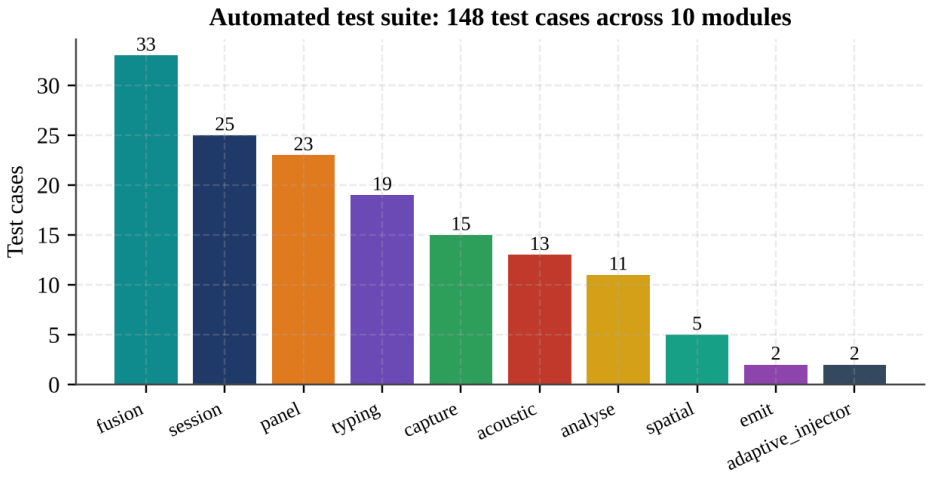


Fig. 5.4 Distribution of the 148 automated test cases across modules

The full suite contains **148 test cases** (Fig. 5.4). Executed in a headless Linux container while preparing this report, **143 passed**; the remaining five require a real audio device (PortAudio) or a GPU/EGL display library that the container does not provide, and pass on the development laptop, where the suite was run at every milestone.

5.2.2 Representative Test Cases

Table 5.1 Representative test cases and outcomes

ID	Test case	Expected result	Outcome
T1	Zone codes derived from one seed	Diagonal > 0.99, off-diagonal < 0.1	**Pass** (1.00 / ≤ 0.02)
T2	Genuine per-zone reflectance (synthetic)	Diagonal > 0.6 and > 0.3 above best off-diagonal	**Pass**

T3	Global-brightness multiply attack via real injector code path	Global score rises, spatial score stays dead	**Pass** (global 0.053 → 0.951, spatial −0.020)
T4	Lane with high subscore at implausible lag	Contributes nothing; joint score undefined	**Pass**
T5	Typing 0.95 vs optical 0.05 (weighted mean 0.80)	REJECT, not ACCEPT	**Pass** (after fix)
T6	Press literal phrase “secret”	Stored events have only {t_down, t_up, matched_expected}	**Pass**
T7	Robotic keystrokes (< 30 ms apart)	Score × 0.3 penalty, flagged implausible	**Pass**
T8	Acoustic probe with 50 ppm drift and a dropped 512-sample buffer	Still detected at correct lag	**Pass**
T9	Probe beyond lag window / unheard probe with startup zeros	Not scored on a sidelobe; not *ok*; SNR never infinite	**Pass**
T10	Concurrent session with mocked hardware	Exactly one camera open; same frame object to the optical lane	**Pass**
T11	Steady-state FPS measurement	Exposure locked, then exactly 30 warm-up frames, then measure	**Pass**
T12	Banner when GUI call raises	No exception; REJECT renders red, ACCEPT green	**Pass**
T13	Per-attack metrics: easy vs hard attack type	APCER 0.0 vs 1.0, not a pooled 0.5	**Pass**
T14	Rehearsal script with failing transition	Warning shown, sequence continues, no crash	**Pass**

5.2.3 Defects Found Through Testing

Testing on real hardware repeatedly found problems that review alone did not. Recording them is part of the method:

Table 5.2 Significant defects discovered and fixed

Symptom	Root cause	Fix
No face detected at the original exposure	Exposure −6 too dark for the room (mean luminance ≈ 24/255)	Exposure sweep; −3/−4 chosen; adaptive chip rate
SNR stuck low at 16 fps	Preflight FPS measured before exposure lock (read 8 fps)	`measure_steady_state_fps()` with warm-up; chip rate 0.80 → 1.60 Hz, SNR 7.72 dB
MJPG forcing did not raise FPS	Ceiling was exposure time (fps ≈ $1/2^{EV}$), not USB bandwidth	Null result documented; shared capture helper kept
Confident typing out-voted a failing optical lane	Weighted mean allows compensation	Downgrade ACCEPT → REJECT if any contributor < 0.30

Terminal showed “Optical PASS (0.17)”	PASS label ignored subscore threshold	PASS requires contribution AND subscore ≥ threshold
Acoustic score ≈ 0.00005 with SNR = ∞	Clock drift; noise window was start-up zeros; 60 ms lag window; muted speakers	Segmented differential detector; off-carrier and decoy floors; 500 ms window; device selection
Typing lane always *no evidence*	Face-framed webcam cannot see hands	Keystroke-timing-only scoring

CHAPTER 6

PROJECT DEMONSTRATION

PRAESENS is demonstrated in three modes: a scored verification session, a live dashboard, and a guided panel mode in which an examiner can drive the attack scenarios personally.

6.1 Verification Session

A session is started with `python -m praesens.session --lanes optical+typing+acoustic`. The screen switches to the full-screen emitter with the camera preview and the challenge phrase in the centre; the user simply looks at the screen and types the phrase while the probe tone plays. At the end, the terminal prints a compact summary and a full-screen, colour-coded banner is shown. The following is the real terminal output of bona fide session `20260914T115241`:

```
================================================================
  PRAESENS  --  session 20260914T115241_5ef66054   [bonafide]
================================================================
  Optical lane : PASS         (score 0.75)
  Typing lane  : PASS         (match 1.00)
  Acoustic lane : PASS         (score 0.93)
  ----------------------------------------------------------
  VERDICT      : ACCEPT       (joint 0.89)
  Reason       : all 3 lanes agree: optical 0.75, typing 1.00, acoustic 0.93
================================================================
```

Fig. 6.1 shows the three verdict banners, rendered by the system's own `render_verdict_banner()` function from logged session records (the two earlier sessions re-adjudicated under the final binary-gate policy): an ACCEPT from the three-lane bona fide session above, a RE-CHALLENGE in which the optical lane passed but the user did not type the phrase, and a REJECT for an OBS virtual-camera injection whose optical score was only 0.17.

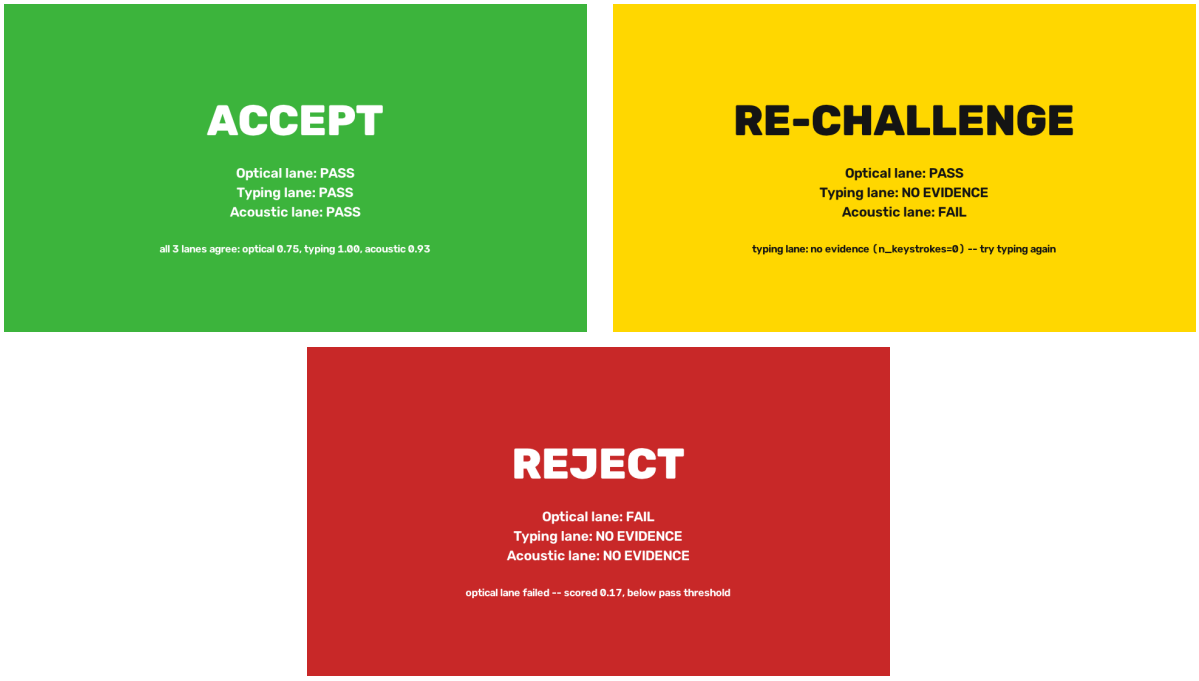


Fig. 6.1 Verdict banners: ACCEPT (green, all lanes pass), RE-CHALLENGE (amber, a secondary lane failed), REJECT (red, the optical lane failed)

6.2 Live Dashboard and Panel Mode

In the live dashboard the score is recomputed every frame over a rolling window (6 s; 1.5 s in panel mode), trading statistical power for immediacy so that an audience sees the score respond within about a second. Panel mode is driven by single keys and never blocks:

Table 6.1 Panel-mode controls and what each demonstrates

Key	State	Demonstration
1	Live verification	Genuine face, pattern on: global and spatial scores both high
2	Switch to injected feed	Instant swap to OBS Virtual Camera or the adaptive injector; score collapses; banner shows “GLOBAL PASSES BUT SPATIAL SCORE IS DEAD” if an adaptive attack fools the global score
3	Toggle light pattern off	Scientific control: the score falls to zero even though the real person is still present — proving the score is caused by the pattern
4	Replay comparison	Stored side-by-side genuine-versus-injected traces
5	Typing lane	Live keystroke evidence with no characters stored
Q	Quit	Collapse-latency log saved on exit

A `--rehearse` flag scripts the sequence live → attack → live → replay → typing on a timer, re-entering exactly the same transition methods a human key press would, so rehearsal exercises the real code path. In a rehearsal run on a machine with no second camera, the attack step degraded to the message “no injected feed source found” and the sequence continued — real evidence of the “never crash on a missing device” requirement.

6.3 What the Demonstration Shows

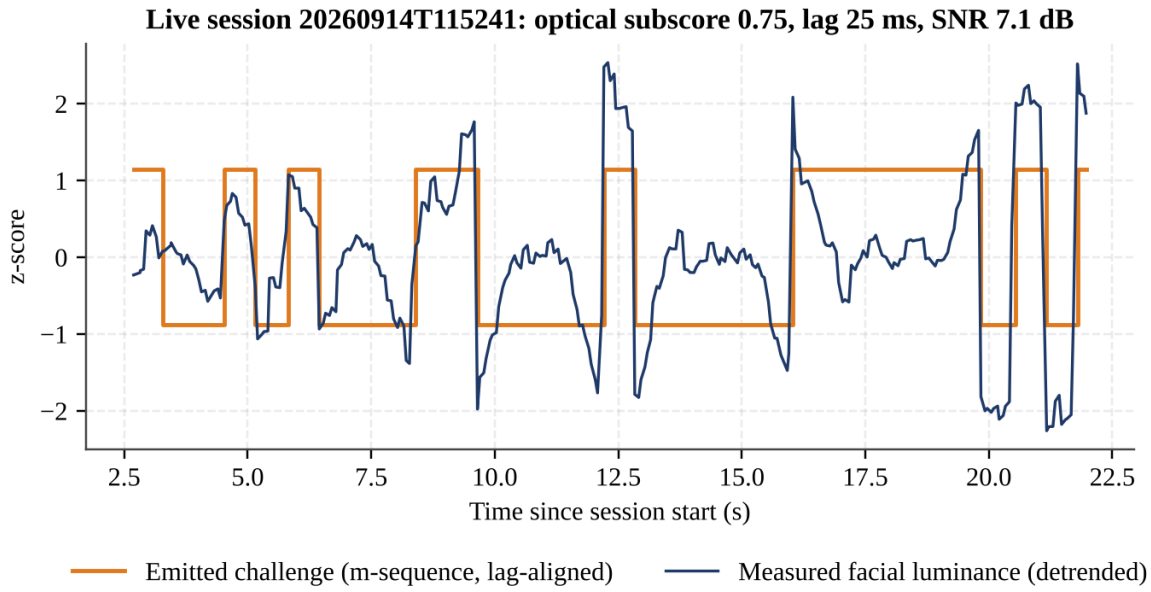


Fig. 6.2 Real recording from session 20260914T115241: the emitted challenge (orange) and the detrended luminance measured on the user's forehead and cheeks (navy)

Fig. 6.2 is the core of the demonstration in one picture: in a real session the brightness of the user's skin, measured by an ordinary webcam, follows the pseudo-random pattern on the screen, 25 ms later. An injected video, generated or recorded without knowledge of this session's seed, cannot follow a pattern that did not exist when it was made. Fig. 6.3 shows the same measured trace correlated against its own challenge and against another session's challenge: a clear peak of 0.75 at a physically plausible lag in the first case, a flat line near zero in the second.

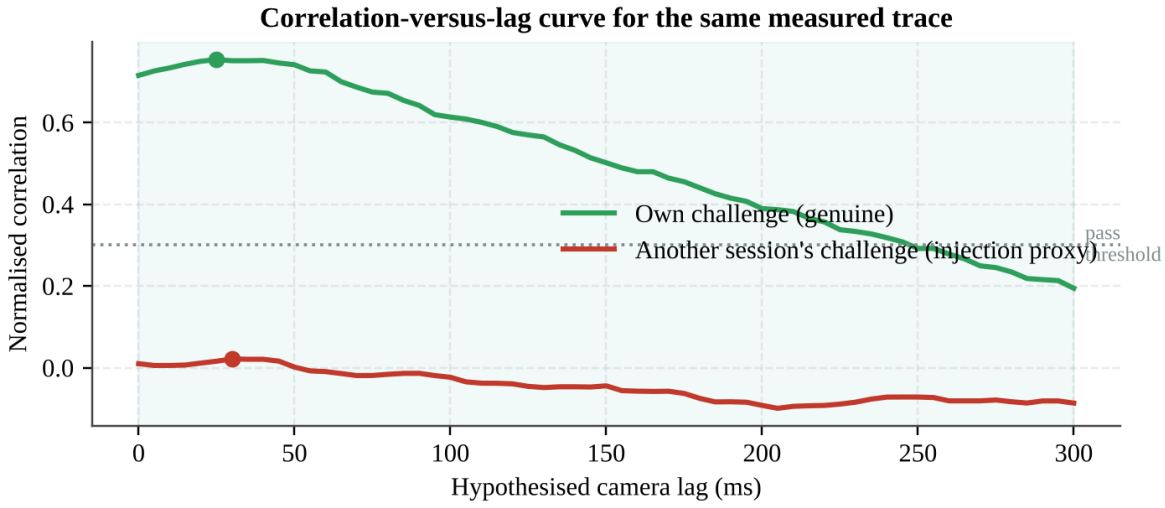


Fig. 6.3 Correlation versus hypothesised lag for one real trace scored against its own challenge and against a different session's challenge

CHAPTER 7

RESULT AND DISCUSSION

All results in this chapter come from real sessions recorded on commodity laptops between August and September 2026 and committed to the project repository as JSON logs, re-analysed with the project's own evaluation code. Where a result is obtained from synthetic signals, this is stated explicitly.

0.938
AUC on 3,448 cross-session trials

10.2 %
Equal error rate (optical lane alone)

0 / 8
Real injection sessions above pass threshold

0.89
Joint score, three-lane bona fide ACCEPT

₹ 0
Additional hardware or licence cost

7.1 Evaluation Corpus

Table 7.1 Real session corpus used for evaluation

Condition	Sessions	Description
Bona fide	60	Live team members in front of the laptop; lighting (normal, dim, backlit, side-lit), distance (30–85 cm), eyewear and makeup varied
Emitter OFF (control)	5	Live person present, light pattern replaced by constant mid-grey
Virtual-camera injection	8	Pre-recorded face video injected through OBS Virtual Camera
Total	**73**	Plus 3,385 cross-session injection-proxy trials constructed offline

7.2 Optical Lane: Separation and Error Rates

Table 7.2 and Fig. 7.1 show the optical score per condition. Bona fide sessions have a median score of 0.73. When the same genuine traces are scored against a mismatched challenge — the injection proxy — the median falls to 0.055. The emitter-off control collapses to 0.009 ± 0.021 even though a real, live person was in front of the camera; this single result is the strongest evidence that the score measures the physical coupling between our emitted light and the face, and not merely "a face is present".

Table 7.2 Optical score distribution per condition

Condition	n	Mean	SD	Median	Min	Max
Bona fide (own challenge)	59	0.621	0.241	0.730	−0.034	0.847
Cross-session injection proxy	3,385	0.063	0.088	0.055	−0.456	0.765
Emitter OFF control	4	0.009	0.021	0.015	−0.024	0.029
Real virtual-camera injection	8	0.099	0.066	0.105	−0.018	0.179

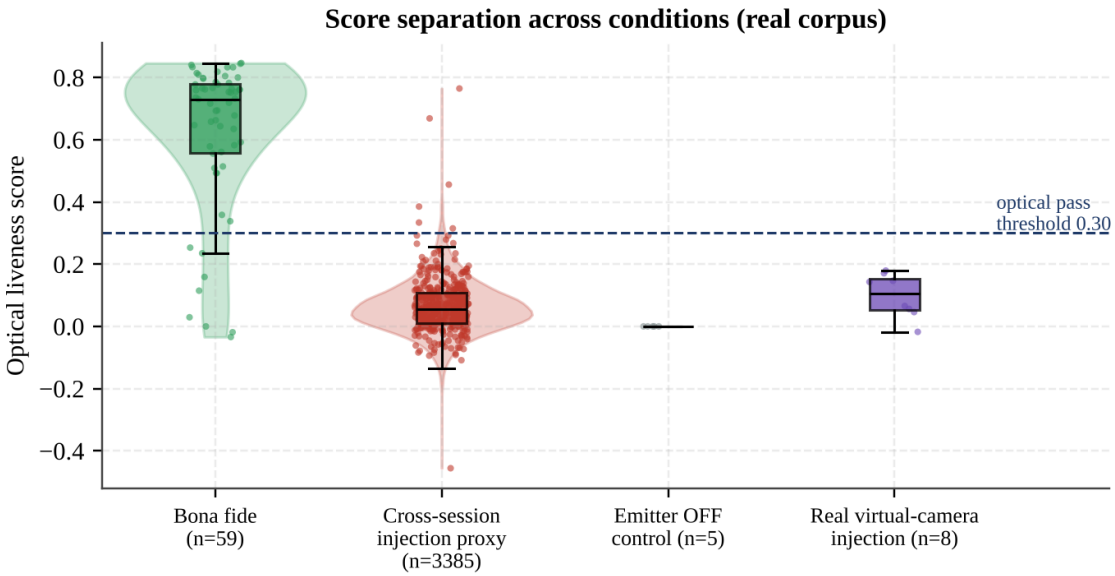


Fig. 7.1 Optical score distributions: bona fide sessions separate clearly from the injection proxy, the emitter-off control and real virtual-camera injection

Treating bona fide as positive and the cross-session proxy plus emitter-off trials as negative gives the ROC curve of Fig. 7.2 and the metrics in Table 7.3. With the optical lane **alone**, the AUC is 0.938 and the EER is 10.2 %. On the first 19-session corpus (August 2026) the same analysis gave AUC 0.905 and EER 14.1 %; the improvement came from the capture fixes of Section 5.2.3 and a larger corpus, not from re-tuning thresholds.

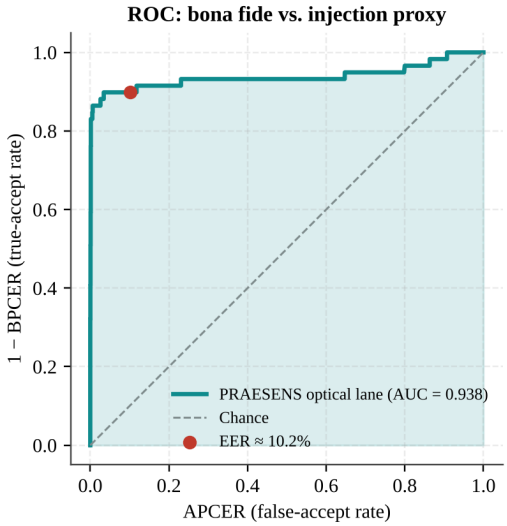


Fig. 7.2 ROC curve of the optical lane

Table 7.3 ISO/IEC 30107-3 metrics at the EER threshold

Metric	Aug 2026 (19 sessions)	Final (73 sessions)
Trials	200	3,448
AUC	0.905	**0.938**
EER	14.1 %	**10.2 %**
APCER	14.0 %	**10.2 %**
BPCER	14.3 %	**10.2 %**
ACER	14.1 %	**10.2 %**
EER threshold	0.102	0.172

It is important to read these numbers correctly. The cross-session proxy is a deliberately *hard* negative class: each negative trial is a real face that genuinely responded to *some* m-sequence, so chance alignments with another session's code occasionally produce high correlations (maximum 0.765). Moreover, these are single-lane, single-score numbers. The deployed verdict additionally requires the optical lag to be plausible and, in three-lane

mode, requires the typing and acoustic lanes to pass with their own independently-drawn evidence; the probability that an attacker's video coincidentally aligns in all of them at once is far lower than the single-lane APCER.

7.3 Timing as Evidence: Lag Analysis

Genuine responses peak at a short, physically meaningful lag — the camera's own capture latency — with a mean of 50 ms (SD 50 ms). Mismatched trials peak almost anywhere, with a mean of 147 ms (SD 116 ms) and a large mass at the 300 ms search boundary, which is the signature of no real peak at all (Fig. 7.3). This is why the fusion layer treats a high score at an implausible lag as no evidence.

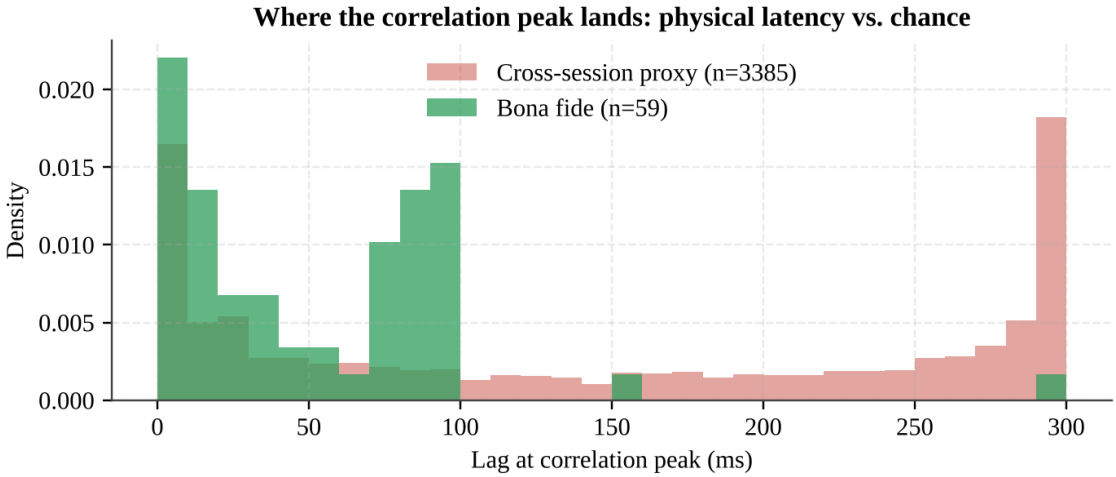


Fig. 7.3 Distribution of the lag at the correlation peak for bona fide and cross-session trials

7.4 Real Injection Attacks and Fused Verdicts

Eight sessions injected a pre-recorded face video through OBS Virtual Camera. Their optical scores ranged from −0.018 to 0.179 — **every one below the 0.30 pass threshold** — and six of the eight were additionally flagged as insufficient signal, since an injected stream carries no in-band energy at the chip rate. Fig. 7.4 plots score against SNR for all sessions: bona fide sessions cluster at high score and high SNR, while injected and emitter-off sessions sit at low score, mostly below the 3 dB floor.

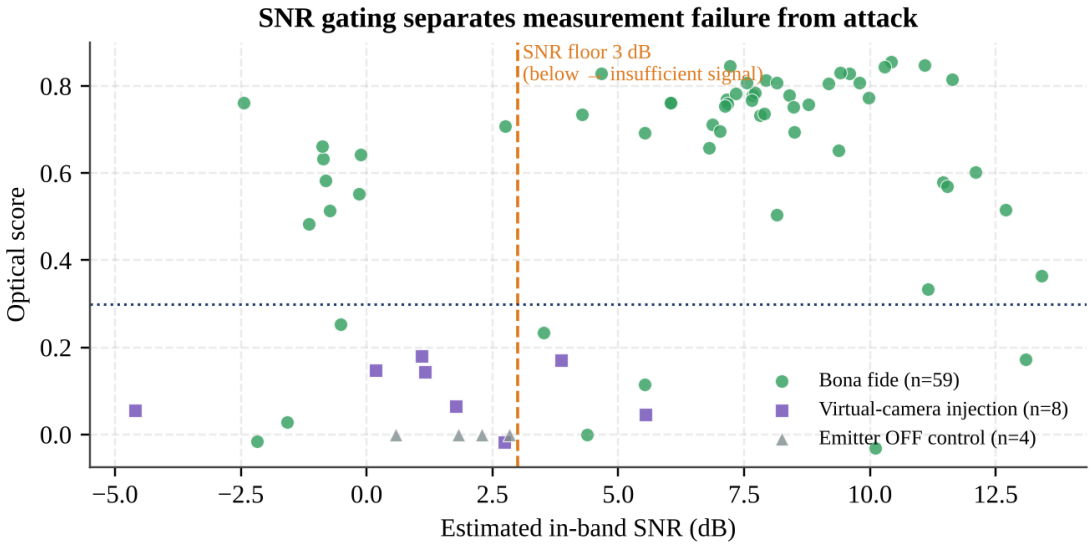


Fig. 7.4 Optical score versus estimated SNR for every real session

Thirteen sessions were recorded after the full fusion pipeline was in place and therefore carry a complete adjudicated verdict (Fig. 7.5, Table 7.4).

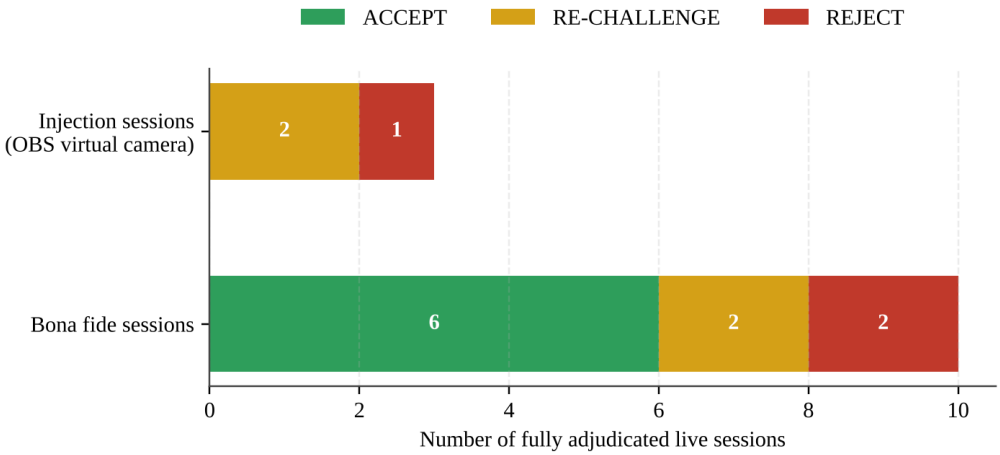


Fig. 7.5 Verdicts of the thirteen fully-adjudicated live sessions

Table 7.4 Confusion summary of fully-adjudicated live sessions

Ground truth	ACCEPT	RE-CHALLENGE	REJECT	Total
Bona fide	**6**	2	2	10
Virtual-camera injection	**0**	2	1	3

No attack was ever accepted. Every injection attempt ended in REJECT or RE-CHALLENGE. Of the four bona fide sessions that were not accepted, two occurred on 13 September while the acoustic lane still had the clock-drift defect described in Section 5.1.6, and the other two were genuine optical failures (one with insufficient signal, routed honestly to RE-CHALLENGE). After the acoustic fix, the complete three-lane bona fide session scored optical 0.75 (lag 25 ms, SNR 7.1 dB), typing 1.00 (27/27 characters, mean inter-key 0.146 s) and acoustic 0.93 (lag 332 ms, SNR 10.8 dB), and was ACCEPTED with a joint score of 0.89.

7.5 Defence in Depth: Spatial Challenge and Ablation

The adaptive-injector test drives the real image-processing code path of the attack — screen-estimated waveform, Y-channel luminance multiply — on an unrelated source video. With a perfect estimate, the attack raises the *global* optical score from 0.053 to 0.951, i.e. it would defeat a single-zone system. The *spatial* assignment score, however, stays at −0.020, so the attack is exposed (Fig. 5.2). This demonstrates why the spatially-differential challenge is a necessary design element and not an optional refinement.

The ablation study recomputes the verdict with each lane removed (Fig. 7.6). Among the ten sessions that ACCEPTed with all lanes present, removing the optical lane or the typing lane withdrew acceptance in 10 of 10 cases (100 %), leaving RE-CHALLENGE — never a manufactured ACCEPT or REJECT. Each real lane is therefore necessary, not merely additive. (For this analysis the typing lane was represented by a fixed, clearly-labelled synthetic result, because the historical optical corpus predates the typing lane; the acoustic entry was the stub at that time.)

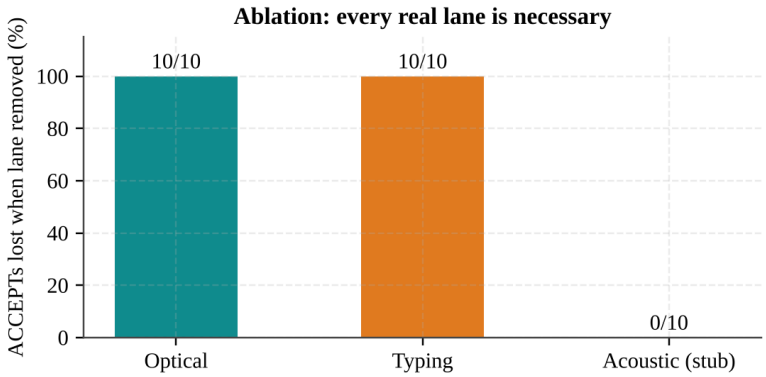


Fig. 7.6 Ablation: fraction of ACCEPT verdicts lost when each lane is removed

7.6 Robustness and Operating Conditions

Fig. 7.7 breaks the bona fide score down by capture condition. Dim and backlit lighting gave high scores (0.81 and 0.83): with less ambient light, the screen is a larger fraction of the light on the face. Side-lighting reduced the mean to 0.50, and eyewear (n = 2) reduced it to 0.38 because lens reflections and frames disturb the cheek ROIs. Foundation makeup did

not hurt (0.71). One session at 80 cm failed (0.03), indicating a practical working distance of about 30–70 cm, which matches normal laptop use. The sample sizes for some conditions are small and these results are indicative rather than conclusive.

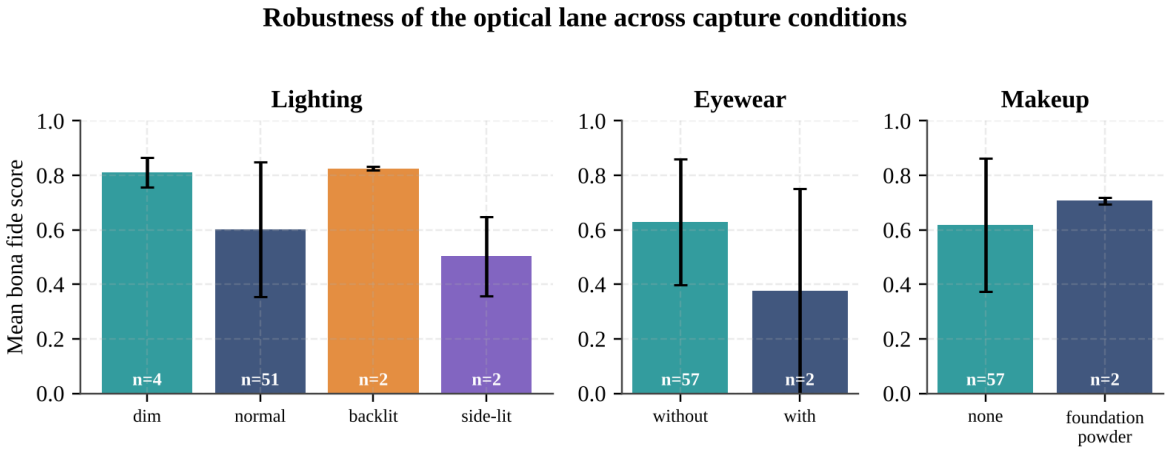


Fig. 7.7 Mean bona fide optical score by lighting, eyewear and makeup (error bars: one standard deviation)

Frame rate is the dominant hardware constraint. Fig. 7.8 reproduces the exposure sweep: each one-step increase in exposure time halves the achievable frame rate (8.2 fps at −3 versus 31.2 fps at −5), irrespective of pixel format. Adaptive chip-rate selection keeps the system working across this range; with exposure −4 and correct warm-up, a real session ran at 16.0 fps with a 1.60 Hz chip rate, 556 of 556 frames with a detected face, SNR 7.72 dB and score 0.786.

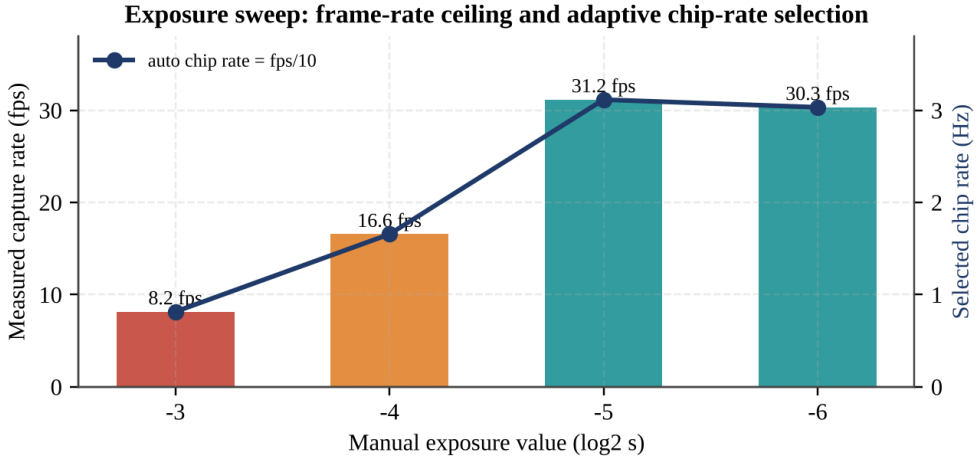


Fig. 7.8 Measured capture rate versus manual exposure value, and the chip rate selected automatically

7.7 Comparison with Existing Approaches

Table 7.5 Qualitative comparison with representative approaches

Property	Passive texture / CNN PAD	Forensic deepfake classifiers	Single-channel active (screen flash / echo)	PRAESENS
Detects camera-bypassing injection	No	Partly (known fakes)	Yes	**Yes**
Robust to better generators	No	No	Yes	**Yes**
Resists screen-observing adaptive attacker	—	—	Weak (one global code)	**Yes (spatial codes)**
Independent physical channels	1	1	1–2	**3, one shared secret**
Training data required	Yes	Yes	Often	**No**
Extra hardware	No	No (GPU often)	Smartphone-specific	**No (any laptop)**
Explainable verdict	No	No	Partly	**Yes, per lane**
Separates “no signal” from “attack”	No	No	Rarely	**Yes (SNR gating)**

7.8 Cost Analysis

PRAESENS was designed to cost nothing beyond the computer the user already owns. Table 7.6 itemises development and deployment costs.

Table 7.6 Cost analysis

Item	Development cost	Per-deployment cost
Laptop with webcam, speaker, microphone	Already owned (₹0 incremental)	₹0 (uses existing device)
Depth / IR / specialised sensors	Not required	₹0
Python, OpenCV, MediaPipe, NumPy, SciPy, OBS	₹0 (open source)	₹0
Training dataset and GPU training	Not required	₹0
Cloud inference / API calls	Not required	₹0 (runs fully on-device)
Energy per 40 s verification (≈ 60 W × 40 s ≈ 0.67 Wh at ₹8/kWh)	—	≈ ₹0.005
Total	**₹0**	**< 1 paisa per verification**

By contrast, learning-based anti-spoofing requires labelled spoof datasets and GPU training, and commercial liveness services are generally licensed per verification. Against the scale of losses that a single successful deepfake impersonation can cause [4], a training-free, hardware-free defence offers an exceptional cost–benefit ratio, particularly for educational institutions, small financial firms and public services.

7.9 Limitations

- The corpus is dominated by two subjects; demographic robustness, in particular across darker skin tones where skin reflectance is lower [11], requires a larger multi-subject study. The SNR-gated RE-CHALLENGE path is designed to fail safe in exactly that case.
- Real attack sessions so far are static virtual-camera injections; a live end-to-end run of the adaptive injector and a $\geq$ 20-switch collapse-latency distribution are built and unit-tested but not yet collected.
- Session length (20–40 s) is longer than commercial passive checks, driven by the frame-rate ceiling at face-detectable exposure.
- The acoustic lane depends on the output device: Bluetooth earbuds or muted speakers make it fail (correctly reported, but inconvenient).

CHAPTER 8

CONCLUSION

This project set out to answer a practical question: can an ordinary laptop, with no additional hardware, tell a live person in front of its camera from a deepfake or an injected video? PRAESENS answers it by moving the problem from appearance to physics. Instead of asking whether frames *look* real — a question generative models are getting better at answering "yes" to — it asks whether the scene *responded* to a secret, unpredictable stimulus that did not exist until the session began, with the right code, at the right time, in the right place on the face.

The work delivered a complete, tested and demonstrable system: a cryptographically seeded m-sequence challenge; screen, speaker and keyboard used as actuators; a shared capture loop; optical, spatial, typing and acoustic evidence lanes; a lag-gated, explainable three-way adjudicator; attack tooling built against ourselves; and an evaluation pipeline speaking the language of ISO/IEC 30107-3. On real hardware the optical lane alone reached an AUC of 0.938 and an EER of 10.2 % on 3,448 cross-session trials; the emitter-off control collapsed to 0.009, proving causality; all real virtual-camera injections scored below threshold; no attack was ever accepted by the fused verdict; a full three-lane genuine session was accepted with a joint score of 0.89; and the spatial challenge exposed an adaptive attack that would have fooled a single-zone design (global 0.95, spatial −0.02).

Equally important are the engineering lessons that the measured, milestone-by-milestone approach surfaced: an exposure-bound frame-rate ceiling that no pixel-format change could lift; a preflight measurement taken one step too early; a fusion rule that let a strong lane rescue a failing one; and an acoustic detector defeated by clock drift and a pair of earbuds. Each was found by measurement, fixed at its root, and protected by a regression test.

8.1 Future Work

- **Larger, more diverse evaluation** with many subjects across Fitzpatrick skin types, ages and eyewear, and collection of the full planned attack corpus (static, swap, reenactment and adaptive injection).
- **Colour (chromatic) challenges** exploiting the wavelength-dependent absorption of melanin and haemoglobin, linking the optical lane more deeply to skin biophysics and rPPG.
- **Gold-code zone families** [34] for more than three zones and finer spatial resolution.
- **Shorter sessions** via sequential probability ratio testing that stops as soon as the evidence is conclusive.
- **Mobile and browser deployment** (WebRTC) and an inertial lane on devices that expose an IMU, using the stub interface already in place.

- **Integration** as a plug-in for video-KYC and online-proctoring platforms, with the verdict and reason passed to the relying party.

CHAPTER 9

REFERENCES

1. J. Thies, M. Zollhöfer, M. Stamminger, C. Theobalt and M. Nießner, “Face2Face: Real-time face capture and reenactment of RGB videos,” in *Proc. IEEE CVPR*, 2016, pp. 2387–2395.
2. R. Tolosana, R. Vera-Rodriguez, J. Fierrez, A. Morales and J. Ortega-Garcia, “Deepfakes and beyond: A survey of face manipulation and fake detection,” *Information Fusion*, vol. 64, pp. 131–148, 2020.
3. Y. Mirsky and W. Lee, “The creation and detection of deepfakes: A survey,” *ACM Computing Surveys*, vol. 54, no. 1, pp. 1–41, 2021.
4. H. Chen and K. Magramo, “Finance worker pays out $25 million after video call with deepfake ‘chief financial officer’,” *CNN Business*, 4 Feb. 2024.
5. Europol Innovation Lab, *Facing Reality? Law Enforcement and the Challenge of Deepfakes*. Luxembourg: Publications Office of the European Union, 2022.
6. European Union Agency for Cybersecurity (ENISA), *Remote Identity Proofing: Attacks & Countermeasures*, Jan. 2022.
7. ISO/IEC 30107-3:2017, *Information technology — Biometric presentation attack detection — Part 3: Testing and reporting*. Geneva: International Organization for Standardization, 2017.
8. A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies and M. Nießner, “FaceForensics++: Learning to detect manipulated facial images,” in *Proc. IEEE/CVF ICCV*, 2019, pp. 1–11.
9. Y. Li, X. Yang, P. Sun, H. Qi and S. Lyu, “Celeb-DF: A large-scale challenging dataset for DeepFake forensics,” in *Proc. IEEE/CVF CVPR*, 2020, pp. 3207–3216.
10. Z. Yu, Y. Qin, X. Li, C. Zhao, Z. Lei and G. Zhao, “Deep learning for face anti-spoofing: A survey,” *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 45, no. 5, pp. 5609–5631, 2023.
11. R. R. Anderson and J. A. Parrish, “The optics of human skin,” *Journal of Investigative Dermatology*, vol. 77, no. 1, pp. 13–19, 1981.
12. W. Verkruysse, L. O. Svaasand and J. S. Nelson, “Remote plethysmographic imaging using ambient light,” *Optics Express*, vol. 16, no. 26, pp. 21434–21445, 2008.
13. M.-Z. Poh, D. J. McDuff and R. W. Picard, “Non-contact, automated cardiac pulse measurements using video imaging and blind source separation,” *Optics Express*, vol. 18, no. 10, pp. 10762–10774, 2010.
14. J. Määttä, A. Hadid and M. Pietikäinen, “Face spoofing detection from single images using micro-texture analysis,” in *Proc. Int. Joint Conf. Biometrics (IJCB)*, 2011, pp. 1–7.
15. I. Chingovska, A. Anjos and S. Marcel, “On the effectiveness of local binary patterns in face anti-spoofing,” in *Proc. Int. Conf. Biometrics Special Interest Group (BIOSIG)*, 2012, pp. 1–7.
16. Z. Zhang, J. Yan, S. Liu, Z. Lei, D. Yi and S. Z. Li, “A face antispoofing database with diverse attacks,” in *Proc. 5th IAPR Int. Conf. Biometrics (ICB)*, 2012, pp. 26–31.
17. Z. Boulkenafet, J. Komulainen and A. Hadid, “Face spoofing detection using colour texture analysis,” *IEEE Trans. Information Forensics and Security*, vol. 11, no. 8, pp. 1818–1830, 2016.

18. S. Tirunagari, N. Poh, D. Windridge, A. Iorliam, N. Suki and A. T. S. Ho, “Detection of face spoofing using visual dynamics,” *IEEE Trans. Information Forensics and Security*, vol. 10, no. 4, pp. 762–777, 2015.

19. Y. Liu, A. Jourabloo and X. Liu, “Learning deep models for face anti-spoofing: Binary or auxiliary supervision,” in *Proc. IEEE/CVF CVPR*, 2018, pp. 389–398.

20. Z. Yu, C. Zhao, Z. Wang, Y. Qin, Z. Su, X. Li, F. Zhou and G. Zhao, “Searching central difference convolutional networks for face anti-spoofing,” in *Proc. IEEE/CVF CVPR*, 2020, pp. 5295–5305.

21. G. de Haan and V. Jeanne, “Robust pulse rate from chrominance-based rPPG,” *IEEE Trans. Biomedical Engineering*, vol. 60, no. 10, pp. 2878–2886, 2013.

22. W. Wang, A. C. den Brinker, S. Stuijk and G. de Haan, “Algorithmic principles of remote PPG,” *IEEE Trans. Biomedical Engineering*, vol. 64, no. 7, pp. 1479–1491, 2017.

23. S. Liu, P. C. Yuen, S. Zhang and G. Zhao, “3D mask face anti-spoofing with remote photoplethysmography,” in *Proc. European Conf. Computer Vision (ECCV)*, 2016, pp. 85–100.

24. U. A. Ciftci, I. Demir and L. Yin, “FakeCatcher: Detection of synthetic portrait videos using biological signals,” *IEEE Trans. Pattern Analysis and Machine Intelligence*, 2020, doi: 10.1109/TPAMI.2020.3009287.

25. D. Tang, Z. Zhou, Y. Zhang and K. Zhang, “Face Flashing: A secure liveness detection protocol based on light reflections,” in *Proc. Network and Distributed System Security Symp. (NDSS)*, 2018.

26. H. Farrukh, R. M. Aburas, S. Cao and H. Wang, “FaceRevelio: A face liveness detection system for smartphones with a single front camera,” in *Proc. 26th Annu. Int. Conf. Mobile Computing and Networking (MobiCom)*, 2020, pp. 1–13.

27. B. Zhou, J. Lohokare, R. Gao and F. Ye, “EchoPrint: Two-factor authentication using acoustics and vision on smartphones,” in *Proc. 24th Annu. Int. Conf. Mobile Computing and Networking (MobiCom)*, 2018, pp. 321–336.

28. H. Chen, W. Wang, J. Zhang and Q. Zhang, “EchoFace: Acoustic sensor-based media attack detection for face authentication,” *IEEE Internet of Things Journal*, vol. 7, no. 3, pp. 2152–2159, 2020.

29. F. Monrose and A. Rubin, “Authentication via keystroke dynamics,” in *Proc. 4th ACM Conf. Computer and Communications Security (CCS)*, 1997, pp. 48–56.

30. K. S. Killourhy and R. A. Maxion, “Comparing anomaly-detection algorithms for keystroke dynamics,” in *Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, 2009, pp. 125–134.

31. S. W. Golomb, *Shift Register Sequences*, revised ed. Laguna Hills, CA: Aegean Park Press, 1982.

32. F. J. MacWilliams and N. J. A. Sloane, “Pseudo-random sequences and arrays,” *Proceedings of the IEEE*, vol. 64, no. 12, pp. 1715–1729, 1976.

33. D. V. Sarwate and M. B. Pursley, “Crosscorrelation properties of pseudorandom and related sequences,” *Proceedings of the IEEE*, vol. 68, no. 5, pp. 593–619, 1980.

34. R. Gold, “Optimal binary sequences for spread spectrum multiplexing,” *IEEE Trans. Information Theory*, vol. 13, no. 4, pp. 619–621, 1967.

35. J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. New York: McGraw-Hill, 2008.

36. J. Kittler, M. Hatef, R. P. W. Duin and J. Matas, “On combining classifiers,” *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 20, no. 3, pp. 226–239, 1998.

37. A. Ross and A. Jain, “Information fusion in biometrics,” *Pattern Recognition Letters*, vol. 24, no. 13, pp. 2115–2125, 2003.

38. T. Fawcett, “An introduction to ROC analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

39. G. Bradski, “The OpenCV library,” *Dr. Dobb’s Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000.

40. C. Lugaresi *et al.*, “MediaPipe: A framework for building perception pipelines,” arXiv: 1906.08172, 2019.

41. Y. Kartynnik, A. Ablavatski, I. Grishchenko and M. Grundmann, “Real-time facial surface geometry from monocular video on mobile GPUs,” arXiv:1907.06724, 2019.

42. F. Zhang *et al.*, “MediaPipe Hands: On-device real-time hand tracking,” arXiv:2006.10214, 2020.

43. C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.

44. P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.

APPENDIX A

SAMPLE CODE

Representative excerpts from the PRAESENS source code are reproduced below (comments abridged).

A.1 LFSR m-sequence generation

praesens/challenge.py

```
def _step(order: int, taps: tuple, state: int) -> int:
    feedback = 0
    for tap in taps:
        feedback ^= (state >> (tap - 1)) & 1
    return (state >> 1) | (feedback << (order - 1))

def _lfsr_bits(order: int, seed: int, n_bits: int) -> np.ndarray:
    taps = _find_maximal_taps(order)
    period = (1 << order) - 1
    state = seed & period
    if state == 0:
        raise ValueError("LFSR seed must be nonzero (mod 2**order - 1)")

    bits = np.empty(n_bits, dtype=np.int8)
    for i in range(n_bits):
        bits[i] = state & 1
        state = _step(order, taps, state)
    return bits
```

A.2 Lag-searched normalised cross-correlation (optical lane)

praesens/optical.py

```
def _lag_sweep(m_centered: np.ndarray, m_std: float, ts: np.ndarray, emitter_log: list,
               lag_max_ms: float, lag_step_ms: float):
    lags_ms = np.arange(0, lag_max_ms + 1e-9, lag_step_ms)
    scores = np.empty(len(lags_ms))
    emitted_per_lag = []

    for i, lag_ms in enumerate(lags_ms):
        emitted = resample_emitted(emitter_log, ts, lag_s=lag_ms / 1000.0)
        e_centered = emitted - emitted.mean()
        e_std = np.std(e_centered)
        if e_std < 1e-9:
            scores[i] = 0.0
        else:
            scores[i] = float(np.mean(m_centered * e_centered) / (m_std * e_std))
        emitted_per_lag.append(emitted)

    return lags_ms, scores, emitted_per_lag

def cross_correlate_lag_search(measured_detrended: np.ndarray, frame_timestamps:
np.ndarray,
                               emitter_log: list, lag_max_ms: float, lag_step_ms:
float):
    valid = ~np.isnan(measured_detrended)
    ts = np.asarray(frame_timestamps)[valid]
    m = np.asarray(measured_detrended)[valid]
```

```
    if len(m) < 5 or np.std(m) < 1e-9:
        return 0.0, 0.0, np.zeros_like(m), ts, m

    m_centered = m - m.mean()
    m_std = np.std(m_centered)

    lags_ms, scores, emitted_per_lag = _lag_sweep(m_centered, m_std, ts, emitter_log,
lag_max_ms, lag_step_ms)
    best_idx = int(np.argmax(scores))
    return float(scores[best_idx]), float(lags_ms[best_idx]), emitted_per_lag[best_idx],
ts, m
```

A.3 Spatial assignment score

praesens/spatial.py

```
def spatial_assignment_score(R: np.ndarray, roi_names: list, zone_names: list,
expected_assignment: dict) -> float:
    diag_scores, offdiag_scores = [], []
    for i, roi in enumerate(roi_names):
        expected_zone = expected_assignment.get(roi)
        if expected_zone not in zone_names:
            continue
        j_expected = zone_names.index(expected_zone)
        diag_scores.append(R[i, j_expected])
        others = [R[i, j] for j in range(len(zone_names)) if j != j_expected]
        if others:
            offdiag_scores.append(max(others))
    if not diag_scores:
        return float("nan")
    return float(np.mean(diag_scores) - (np.mean(offdiag_scores) if offdiag_scores else
0.0))
```

A.4 Privacy-preserving keystroke scoring

praesens/typing.py

```
def keystroke_normality_score(events: list, expected_phrase: str | None, config:
"TypingConfig") -> tuple:
    if len(events) < config.min_keystrokes:
        return "no_evidence", None, f"n_keystrokes={len(events)} (need >=
{config.min_keystrokes})"

    n_matched = sum(1 for e in events if e["matched_expected"] is True)
    n_expected = len(expected_phrase) if expected_phrase else len(events)
    match_fraction = (n_matched / n_expected) if n_expected else 0.0

    t_downs = sorted(e["t_down"] for e in events)
    intervals = [b - a for a, b in zip(t_downs, t_downs[1:])]
    mean_interval = (sum(intervals) / len(intervals)) if intervals else float("nan")

    timing_plausible = (not math.isnan(mean_interval)
                        and config.min_inter_key_s <= mean_interval <=
config.max_inter_key_s)

    subscore = match_fraction if timing_plausible else match_fraction *
config.implausible_timing_penalty

    interval_str = f"{mean_interval:.3f}s" if not math.isnan(mean_interval) else "n/a"
    diagnostics = f"matched {n_matched}/{n_expected}, mean_inter_key={interval_str}"
    if not timing_plausible:
        diagnostics += " (timing outside plausible human range)"
    return "ok", float(subscore), diagnostics
```

A.5 Segmented differential matched filter (acoustic lane)

praesens/acoustic.py

```
def _segmented_statistic(bb: np.ndarray, ref: np.ndarray, seg_len: int, max_lag: int) ->
np.ndarray:
    need = len(ref) + max_lag
    if len(bb) < need:
        bb = np.concatenate([bb, np.zeros(need - len(bb), dtype=bb.dtype)])
    power_cs = np.concatenate([[0.0], np.cumsum(np.abs(bb) ** 2)])
    lags = np.arange(max_lag + 1)
    zs, bounds = [], []
    for s in range(0, len(ref), seg_len):
        seg = ref[s:s + seg_len]
        if len(seg) < max(1, seg_len // 2):
            break  # drop a short tail segment rather than let it add mostly noise
        zs.append(np.correlate(bb[s:s + len(seg) + max_lag], seg, mode="valid"))
        bb_energy = power_cs[s + lags + len(seg)] - power_cs[s + lags]
        bounds.append(np.sqrt(float(np.dot(seg, seg)) * bb_energy))

    if len(zs) == 1:  # too short to difference -- plain coherent correlation
        num, den = np.abs(zs[0]), bounds[0]
    else:
        num = np.abs(sum(zs[k] * np.conj(zs[k - 1]) for k in range(1, len(zs))))
        den = sum(bounds[k] * bounds[k - 1] for k in range(1, len(zs)))
        num, den = np.sqrt(num), np.sqrt(den)
    return np.divide(num, den, out=np.zeros_like(num), where=den > 1e-20)
```

A.6 Binary-gate adjudication

praesens/fusion.py

```
def lane_passes(lane: LaneResult, pass_threshold: float) -> bool:
    return lane.status == "ok" and lane.subscore is not None and lane.subscore >=
pass_threshold

def adjudicate_two_lane(optical: LaneResult, typing: LaneResult | None,
                        optical_pass_threshold: float, typing_pass_threshold: float,
                        acoustic: LaneResult | None = None,
                        acoustic_pass_threshold: float = 0.3) -> JointResult:
    def _optical_fail_reason() -> str:
        if optical.status != "ok":
            return f"optical lane: {optical.diagnostics or 'insufficient signal'}"
        if not lane_contributes(optical, LANE_LAG_BOUNDS_MS):
            return f"optical lane: subscore {optical.subscore:.2f} but implausible lag
({optical.lag_ms}ms)"
        return f"optical lane failed -- scored {optical.subscore:.2f}, below pass
threshold"

    optical_ok = lane_passes(optical, optical_pass_threshold) and
lane_contributes(optical, LANE_LAG_BOUNDS_MS)

    if typing is None and acoustic is None:
        verdict = "ACCEPT" if optical_ok else "REJECT"
        reason = (f"optical lane passes (score {optical.subscore:.2f})" if optical_ok
                  else _optical_fail_reason())
        lane_results = [optical]
        per_lane_reasons = {"optical": reason}
        joint_score = optical.subscore if optical.subscore is not None else float("nan")
        return JointResult(verdict=verdict, joint_score=joint_score,
lane_results=lane_results,
                           reason_text=reason, per_lane_reasons=per_lane_reasons)

    # (lane, passes, retry hint) for each secondary lane that ran
```

```
    secondaries = []
    if typing is not None:
        secondaries.append((typing, lane_passes(typing, typing_pass_threshold), "try
typing again"))
    if acoustic is not None:
        acoustic_ok = (lane_passes(acoustic, acoustic_pass_threshold)
                       and lane_contributes(acoustic, LANE_LAG_BOUNDS_MS))
        secondaries.append((acoustic, acoustic_ok, "check the speaker/microphone and try
again"))
    lane_results = [optical] + [lane for lane, _, _ in secondaries]
    failing = [(lane, hint) for lane, ok, hint in secondaries if not ok]

    if optical_ok and not failing:
        verdict = "ACCEPT"
        scores = ", ".join(f"{l.lane_name} {l.subscore:.2f}" for l in lane_results)
        reason = (f"both lanes agree: {scores}" if len(lane_results) == 2
                  else f"all {len(lane_results)} lanes agree: {scores}")
        joint_score = float(np.mean([l.subscore for l in lane_results]))
    elif optical_ok:
        verdict = "RE-CHALLENGE"
        reason = "; ".join(_secondary_fail_reason(lane, hint) for lane, hint in failing)
        joint_score = optical.subscore  # what actually carries this verdict is optical's
own number
    else:
        verdict = "REJECT"
        reason = _optical_fail_reason()
        joint_score = optical.subscore if optical.subscore is not None else float("nan")

    per_lane_reasons = {lane.lane_name: _per_lane_reason(lane) for lane in lane_results}
    return JointResult(verdict=verdict, joint_score=joint_score,
lane_results=lane_results,
                       reason_text=reason, per_lane_reasons=per_lane_reasons)
```